

A First Look at Financial Data Analysis Using ChatGPT-4o*

Zifeng Feng

The University of Texas at El Paso
500 W. University Ave, El Paso, TX 79968, USA
Email: zfeng@utep.edu

Bingxin Li

West Virginia University
83 Beechurst Avenue, Morgantown, WV 26505, USA
Email: bingxin.li@mail.wvu.edu

Feng Liu

The University of Texas at El Paso
500 W. University Ave, El Paso, TX 79968, USA
Email: flu@utep.edu

Current Version: May 30, 2024

Abstract:

OpenAI's new flagship model, ChatGPT-4o, released on May 13, 2024, offers enhanced natural language understanding and more coherent responses. In this paper, we investigate ChatGPT-4o's capabilities in financial data analysis, including zero-shot prompting, time series analysis, risk and return analysis, and ARMA-GARCH estimation. We find that ChatGPT-4o's performance is generally comparable to traditional statistical software like Stata, though some errors and discrepancies arise due to differences in implementation. Despite these issues, our findings indicate that ChatGPT-4o has significant potential for real-world financial analysis. Integrating ChatGPT-4o into financial research and practice may lead to more efficient data processing, improved analytical capabilities, and better-informed investment decisions.

Keywords: ChatGPT, large language models, AI, finance research, academia, data analysis, stock return.

JEL Code: G00, C89, O33

* Acknowledgements: The writing was polished by ChatGPT-4o and Grammarly.

1. Introduction

Released on May 13, 2024, OpenAI's new flagship model, ChatGPT-4o (with its "o" standing for "omni"), represents a significant leap forward in artificial intelligence, capable of reasoning across audio, vision, and text in real-time. Compared to previous iterations, ChatGPT-4o markedly enhances natural language understanding and generation, providing more coherent and contextually relevant responses. This model boasts an expanded knowledge base with more recent and specialized information, particularly in fields like finance and healthcare. It also offers advanced analytical capabilities, improved interactivity, better handling of ambiguities, and increased personalization, making it a versatile tool for various professional applications.

The release of ChatGPT-4o has garnered widespread praise from various industries and the media. According to OpenAI, GPT-4o has shown superior capabilities across text, vision, and audio processing, making it twice as fast and 50% cheaper than its predecessor, GPT-4 Turbo. The model has achieved high ELO scores in benchmarks, significantly outperforming previous models in tasks involving multilingual comprehension, speech recognition, translation, and visual perception.¹ These advancements underline ChatGPT-4o's potential to revolutionize how industries approach data processing and analysis, setting new standards for AI performance.

As with each new model release, the launch of ChatGPT-4o is expected to prompt a wave of quick tests from experts, especially in mathematics and computer science, to validate the model's performance. The finance industry, in particular, stands to benefit from the model's advanced data analysis capabilities. Given the complexities and dynamic nature of financial data, ChatGPT-4o's ability to provide precise and insightful analysis can be a game-changer for

¹ [OpenAI's new multimodal "GPT-4 omni" combines text, vision, and audio in a single model \(the-decoder.com\)](https://the-decoder.com/news/openai-gpt-4-omni)

researchers and practitioners. The versatility of ChatGPT-4o promises substantial improvements in financial research and practice.

Understanding what ChatGPT-4o excels at is crucial for determining its potential applications in finance. The model's enhanced natural language processing capabilities make it adept at interpreting and generating detailed financial reports, analyzing market trends, and assisting in decision-making processes. Its ability to handle vast amounts of data and generate accurate predictions is particularly valuable in finance, where timely and reliable information is critical. Despite the extensive literature on data analysis using previous versions of ChatGPT (see the literature review in Section 2), no studies have yet explored the capabilities of ChatGPT-4o in this domain. This gap exists because the model is still new, and its full potential remains untapped. This paper aims to be the first to investigate the use of ChatGPT-4o for data analysis in finance, highlighting its strengths and identifying areas for further improvement.

In this paper, we conduct a comprehensive evaluation of ChatGPT-4o's performance in financial data analysis, utilizing empirical daily data from CRSP and Fama French Factors. We undertake various tests to assess the model's capabilities, including zero-shot prompting, time series analysis, risk and return analysis, and ARMA-GARCH estimation. Our empirical results demonstrate ChatGPT-4o's superior performance in financial data analysis. The model excels in interpreting complex datasets and offering insightful analyses that can inform investment strategies and risk management.

Through detailed studies and comparative analyses with results from Stata, we show that ChatGPT-4o accurately identifies key variables, performs robust statistical analyses, and generates useful financial metrics such as Sharpe ratios and market betas. These tests underscore the model's potential to transform financial research methodologies. ChatGPT-4o's ability to handle large

datasets and generate comprehensive analyses efficiently enhances the accuracy and speed of data-driven decision-making processes.

Furthermore, our comparative analyses reveal that ChatGPT-4o's performance is comparable to traditional statistical software like Stata, with minor discrepancies primarily due to differences in implementation methods. This validation highlights the model's reliability and practical utility in real-world financial scenarios. The implications of these findings are significant, suggesting that integrating ChatGPT-4o into financial research and practice can lead to more efficient data processing, improved analytical capabilities, and better-informed investment decisions.

The contributions of this paper are twofold. First, it provides a comprehensive evaluation of ChatGPT-4o's capabilities in the context of financial data analysis, filling a critical gap in the current literature. This evaluation offers a thorough understanding of the model's strengths and limitations in financial data analysis. Second, it highlights the practical implications of adopting this advanced model in finance, offering valuable insights for researchers and practitioners. These implications cover various aspects of financial operations, including market analysis, portfolio management, and risk assessment, thereby demonstrating the model's broad applicability.

Looking ahead, the implications of ChatGPT-4o in future research are profound. Its ability to integrate and analyze data across multiple modalities opens new avenues for interdisciplinary studies, combining insights from finance, economics, computer science, and other fields. The model's advanced capabilities can drive innovation in financial research, leading to more accurate models, improved decision-making processes, and a deeper understanding of market dynamics. Furthermore, the model's adaptability to different data types and contexts suggests potential for future enhancements and applications beyond the current scope of this study. This paper lays the

groundwork for future studies, exploring the vast potential of ChatGPT-4o and setting the stage for its widespread adoption in finance and beyond.

2. Related Literature

Many early papers focused on the general applications of ChatGPT across broad disciplines. For instance, the literature discusses ChatGPT's capacity in scientific writing (Alkaissi and McFarlane, 2023; González-Padilla, 2022; Hosseini et al., 2023) and potential applications in academia and industry (Dale, 2021; Alshater, 2022; Dai et al., 2023; Lin, 2023; Rahman and Watanobe, 2023). It addresses concerns raised by academia (da Silva, 2023; Frye, 2022; Gao et al., 2022; Khalil and Er, 2023; Stokel-Walker, 2023; Thorp, 2023), highlights the limitations of ChatGPT (Borji, 2023; Liu et al., 2023; Shen et al., 2023; Zhu et al., 2023; Rice et al., 2024), and imposes standards to eliminate ethical issues and bias in the implementation of ChatGPT (Anders, 2023; Liebreinz, 2023; Lund and Wang, 2023; Yeo-The and Tang, 2023). These studies emphasize the need for validation and human oversight to ensure accuracy and reliability in its outputs. While ChatGPT can significantly enhance efficiency and depth in research, there is a caution against over-reliance on AI-generated analyses.

Another stream of literature focuses on the importance of interdisciplinary collaboration to maximize the benefits of ChatGPT and similar technologies (e.g., Bang et al., 2023; Van Dis et al., 2023; Zhu et al., 2023; Fahad, Salloum, and Shaalan, 2024; Rice et al., 2024). These works underline the potential for AI to drive innovation across various fields.

The adoption of ChatGPT in business disciplines has garnered significant attention, with numerous studies highlighting its transformative potential. Specifically, in finance, ChatGPT has been utilized for various applications, ranging from general financial analysis to more specific

areas. Examples in general financial analysis include Alshater (2022), Dowling and Lucey (2023), Chen, Wu, and Zhao (2023), Zaremba and Demir (2023), and more,² who discuss the potential applications of ChatGPT in financial data processing and interpretation, modeling and simulation, hypothesis generation, automated report generation, and even drafting research papers. These studies demonstrate how ChatGPT can streamline and improve efficiency in these areas.

Ali and Aysan (2023) explore how ChatGPT's capabilities can transform various financial services, including customer service automation, fraud detection, and personalized financial advice, highlighting the model's ability to enhance efficiency and accuracy in financial operations. Bhatia et al. (2024) introduce FinTral, a family of GPT-4 level multimodal financial large language models, to handle a variety of financial data inputs, including text and numerical data, to improve the accuracy and utility of financial analysis and forecasting.

Specific financial areas also see growing interest in the application of ChatGPT. For instance, Jha et al. (2023) investigate its role in corporate finance, particularly in automating and enhancing decision-making processes. Smales (2023) examines the impact of ChatGPT on interpreting and responding to monetary policy announcements, revealing its potential to improve market efficiency. Li et al. (2024) study the model's ability to identify optimistic biases in human financial analysts, offering insights into how AI can mitigate subjective biases in financial analysis.

Further applications include analyzing stock price movements (Lopez-Lira and Tang, 2024), evaluating stock-picking strategies (Pelster and Val, 2024), and conducting sentiment analysis (Fatouros et al., 2023). Additionally, Aldridge (2023) demonstrates the utility of ChatGPT in performing linear regression analysis, highlighting its versatility in handling complex statistical tasks within finance. Cao and Zhai (2023) analyze the impact of ChatGPT on financial research

² See, for example, Khan and Umer (2024); Shue et al. (2023); Yue et al. (2023).

domains such as ESG, corporate culture, and Federal Reserve opinion analysis. Kim et al. (2024) discuss the application of large language models in analyzing financial statements.

Beyond business disciplines, ChatGPT has shown remarkable promise in fields that require rigorous mathematical analysis. For example, Frieder et al. (2023) evaluate the mathematical reasoning abilities of ChatGPT versions and GPT-4 using datasets specifically curated for graduate-level mathematics. This study highlights strengths in basic mathematical tasks while pointing out limitations in more advanced, graduate-level problem-solving. Korinek (2023) explores the application of large language models like ChatGPT in various domains and details how ChatGPT can assist with mathematical derivations, data analysis, coding, and other research tasks, significantly boosting productivity for researchers.

Recent literature investigates ChatGPT's new features in interpreting visual information. Yang et al. (2023) investigate the model's performance in integrating visual and textual information, highlighting its strengths in understanding and generating descriptions of images, recognizing objects, and interpreting visual data. The authors also identify areas where the model struggles, such as in handling complex visual scenes and nuances in image context. Wang et al. (2024) evaluate the capabilities of ChatGPT in interpreting scientific figures in their article published in NPJ Precision Oncology. They highlight ChatGPT's strengths in recognizing and explaining various plot types effectively, which can aid researchers in understanding complex data visualizations.

Overall, the literature highlights ChatGPT's extensive applications across various disciplines, underscoring its potential to enhance both business and scientific research through advanced mathematical analysis and data processing capabilities.

Our paper fits into the literature by providing a comprehensive evaluation of ChatGPT-4o's capabilities, specifically in financial data analysis. By examining its performance in various tasks, such as zero-shot prompting, time series analysis, risk and return analysis, and ARMA-GARCH estimation, we fill a critical gap in the current literature and highlight the practical implications of integrating ChatGPT-4o into financial research and practice.

3. Data

The financial data for the thirty companies included in the Dow Jones Industrial Average (DJIA) are sourced from the Center for Research in Security Prices (CRSP). For each Dow 30 stock from 2019 to 2023, we collect its permno, date, ticker, comnam, permco, cusip, vol, ret, bid, ask, shrout, numtrd, and retx, along with the CRSP return (including all distributions) Value-Weighted Index (vwret) and the S&P 500 Composite Index Return (sprtrn), at daily frequency. This comprehensive dataset ensures that we capture a detailed and accurate picture of each company's stock performance over the specified period. To calculate the Sharpe ratio and market beta of each stock, data on the daily risk-free rate and the Fama-French three factors are sourced from Kenneth French's website.³ These variables are essential for performing robust financial analysis and for understanding the underlying risk and return dynamics of the stocks.

Table 1 reports the structure of the data used in the empirical tests. Panel A lists the 30 components of the DJIA as of May 15, 2024, collected from CNBC.⁴ Panel B presents the data structure for the "Dow 30 Daily Returns" file, which consists of panel data encompassing 30 firms over five years. This structure allows for a detailed examination of stock performance over time and across different companies. Panel C details the data structure for the "FamaFrench_3Factors"

³ https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html

⁴ <https://www.cnbc.com/dow-30/>

file, noting that we divide the original data by 100 to align with the CRSP data in the analysis. This adjustment ensures consistency and comparability across different data sources, facilitating more accurate and meaningful analyses.

[Table 1 here]

4. Zero-Shot Prompting Analysis

Our analysis begins with zero-shot open-end prompting, which means that the prompt used to interact with ChatGPT-4o does not contain examples or demonstrations but asks it to generate results as to what should be the best approach. One of the advantages of zero-shot open-end prompting is that it may reveal unseen problems and provide unexpected solutions.

After uploading the "Dow 30 Daily Returns" File, which is illustrated in Panel B of Table 1, we ask ChatGPT-4o: "Act as a finance professor and statistician. Analyze this data for the daily returns of Dow 30 components from 2019 to 2023, pulled from the Center for Research in Security Prices (CRSP) via the Wharton Research Data Services (WRDS) platform. Perform analysis that can be used in academic papers. Illustrate the meaning and implications of the results."

In Panel A of Table 2, ChatGPT-4o begins its response by identifying the variables/column headers of the input data. Out of 15 columns, ChatGPT-4o correctly identifies 14 header names. The only misidentified column is "*sprtrn*", which should represent the S&P 500 Composite Index Return but is labeled as "Spread Return" in the responses.

ChatGPT-4o suggests that the statistical analysis should include five parts: descriptive statistics, distribution analysis, time series analysis, risk and return analysis, and correlation analysis. Next, ChatGPT-4o provides further details on what those analyses should entail. In implementing these analyses, ChatGPT-4o starts with descriptive statistics and distribution

analysis. ChatGPT-4o mentions that the descriptive statistics should include the count, mean, standard deviation, minimum, 25th, 50th, and 75th percentiles, maximum, skewness, and kurtosis, without providing the actual numbers for these statistics. For the distribution statistics, ChatGPT-4o suggests including a histogram, KDE plot, and QQ plot. In our example, it plots the distribution of the daily returns, QQ plots of the daily returns, and the time series of the daily returns for MSFT. In the subsequent Time Series Analysis, ChatGPT-4o uses the time series plot for Microsoft's daily returns as an example to illustrate return fluctuations over time. It suggests examining four implications: risk and return, distribution properties, volatility clustering, and modeling returns (such as using the GARCH model).

ChatGPT-4o further proceeds with the analysis, including calculating annualized metrics (return, volatility, and Sharpe ratio), examining correlations, and performing additional time series analysis. In the Correlation Analysis, ChatGPT-4o provides a correlation matrix of daily returns for the Dow 30 components. It identifies stock pairs with high and low correlations and suggests implications for portfolio diversification, risk management, and investment strategy.

Overall, ChatGPT-4o demonstrates a robust understanding of financial data analysis and suggests comprehensive steps to analyze the Dow 30 daily returns. It accurately identifies most data columns and outlines essential descriptive and time series analyses. However, it falls short in providing actual descriptive statistics and mislabels one column, indicating room for improvement in accuracy and completeness. The suggested analyses include calculating annualized metrics, examining correlations, and performing additional time series analysis, which are crucial for understanding the underlying risk and return dynamics. ChatGPT-4o's ability to identify high and low correlated stocks and suggest implications for portfolio diversification, risk management, and

investment strategy highlights its potential utility in financial research, despite some shortcomings in precision.

[Table 2 here]

5. Summary Statistics

Summary statistics are typically reported in the first table in many finance papers. Descriptive statistics is also the first analysis we should conduct according to the Zero-Shot Prompting Analysis by ChatGPT-4o, as illustrated in the previous section. Therefore, we proceed our analysis to compare the summary statistics generated from statistical software with those obtained from ChatGPT-4o prompts.

Table 3 reports the summary statistics for the "Dow 30 Daily Returns" file. We first present the results generated using Stata in Panel A of Table 3. As shown in Panel B, we upload the "Dow 30 Daily Returns" file and ask ChatGPT-4o: "Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS. Provide a summary statistic table that includes the mean, median, standard deviation, minimum, maximum, and number of observations for the following variables: return (ret), volume (vol), bid price (bid), ask price (ask), shares outstanding (shrout), and value-weighted return (vwretd)."

ChatGPT-4o responds with the statistics it calculates. Interestingly, the statistics for return (ret) and volume (vol) do not match those generated by Stata, while those for bid price (bid), ask price (ask), shares outstanding (shrout), and value-weighted return (vwretd) do match. This discrepancy suggests that while ChatGPT-4o can accurately process and analyze most of the data, certain variables might be prone to errors, warranting further investigation and validation.

Overall, ChatGPT-4o provides a reasonably accurate set of summary statistics, demonstrating its utility in performing preliminary data analysis. However, the mismatches in key variables like return and volume highlight the need for cross-validation with traditional statistical software to ensure the accuracy and reliability of the results. These findings underscore the potential of integrating AI tools in financial research while also emphasizing the importance of careful verification.

[Table 3 here]

6. Plotting and Insights

In Table 2, the response of ChatGPT-4o suggests that time series analysis should follow the descriptive and distribution statistics, and it plots the time series of daily returns for MSFT (Microsoft Corp). Therefore, we assess the ability of ChatGPT-4o to generate time series figures and conduct time series analysis (the latter is discussed in the regression analysis section, i.e., fitting an ARMA-GARCH model).

Table 4 evaluates a time series plot for Microsoft's return (ret) and the market's return (vwretd) from 2019 to 2023. Panel A shows the results generated using Stata, illustrating that Microsoft's returns are more volatile than the market's returns, with increased volatility during late 2019 and early 2020.

As shown in Panel B, we upload the "Dow 30 Daily Returns" file and send the following prompts to ChatGPT-4o: "Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS. Provide a time series plot for the return (ret) of Microsoft (ticker = MSFT) and the market return (vwretd)."

Interestingly, ChatGPT-4o has an issue with producing a time-series plot, although it does have the ability to generate a time series plot, as evidenced by Table 2. However, here, instead of generating figures, ChatGPT-4o provides a textual description of the data trends for the daily returns of Microsoft and the market return from 2019 to 2023. For Microsoft, it highlights the company's typical fluctuations associated with a high-profile technology company. ChatGPT-4o notes that the daily return volatility reflects underlying market conditions, including periods of heightened uncertainty like the COVID-19 pandemic. ChatGPT-4o also mentions a positive correlation between Microsoft's returns and the market returns, which was not requested but adds value to the analysis.

Overall, ChatGPT-4o shows potential in analyzing and describing time series data but faces challenges in generating visual representations directly. This limitation underscores the importance of using traditional statistical software or tools for precise visualizations. The descriptive insights provided by ChatGPT-4o demonstrate its analytical capabilities, yet the inconsistencies in generating figures highlight the need for further improvements in its application.

[Table 4 here]

7. Risk and Return Analysis

In the previous two sections, both the results from Stata and the response from ChatGPT-4o indicate increased return volatility during the onset of the COVID-19 period. Following this observation, we examine stock returns around the COVID-19 lockdown. In Table 5, we aim to extract the daily returns of Dow 30 components before and after the first COVID-19 lockdown day (Sunday, March 15, 2020). Specifically, we need to obtain the daily returns on March 13 and

March 16, the sum of these two daily returns for each stock, and the average returns of all stocks on each day.

After uploading the "Dow 30 Daily Returns" file, we command ChatGPT-4o: "Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS. Provide the return (ret) of each stock on the trading days before and after the first COVID lockdown day (Sunday, March 15, 2020). Compute the total return of each stock's two trading days, and the average return of all stocks' two trading days."

ChatGPT-4o does not provide the returns directly but instead supplies step-by-step Python codes as a response. We need to copy and run the suggested code in Python. However, we only obtain partial results using the given Python code, specifically, the sum of the two daily returns for each stock, as shown in the first table in Panel C.

Upon reviewing the code, we find that it successfully extracts the daily returns for each stock from the original dataset but fails to display them. This oversight prevents the full visibility of the individual daily returns on March 13 and March 16. To resolve this issue, we add a few lines of codes to ensure that both individual daily returns and their sums are displayed. The final results, incorporating these changes, are shown in the second table in Panel C.

To ensure the accuracy of the results provided by ChatGPT-4o, we perform a parallel analysis using Stata. The results generated by Stata are presented in Panel A. By comparing these results with those obtained from the modified Python code provided by ChatGPT-4o, we find that they are identical, confirming the reliability of the Python code after our adjustments.

Thus, overall, ChatGPT-4o demonstrates a strong capability in generating effective Python code for financial data analysis. However, users must be vigilant as the initial code may contain minor omissions or errors that require careful verification and potential adjustments.

[Table 5 here]

In Table 2, the response from ChatGPT-4o suggests that we should analyze the risk and return of each stock and compute their annualized metrics. Specifically, it mentions three key metrics: Annualized Return, Annualized Volatility, and Sharpe Ratio.

We focus on the Sharpe Ratio as it involves a multi-step calculation that tests the computation abilities of ChatGPT-4o. To obtain the Sharpe Ratio, we first need to compute the daily excess return (return minus risk-free rate), aggregate these to obtain the annual excess return, compute the annualized stock return volatility, and then use the annualized excess return to divide the annual return volatility. If ChatGPT-4o calculates the Sharpe Ratio correctly, it indicates it can accurately compute the preceding annualized return and volatility.

After uploading the "Dow 30 Daily Returns" file, we command ChatGPT-4o: "Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS, and the daily risk-free rate, pulled from Kenneth R. French - Data Library. Compute the annualized excess return (raw return – risk-free rate) of each stock. Compute the standard deviation of excess returns for each stock each year. Then, calculate the Sharpe Ratio, the risk-adjusted return, for each stock each year. Report the Sharpe Ratio for each stock each year and the average Sharpe Ratio of all stocks each year."

Again, ChatGPT-4o does not provide the results directly but instead supplies step-by-step solutions as a response. ChatGPT-4o suggests this step-by-step approach: (1) Load and merge the data. (2) Compute the annualized excess returns. (3) Calculate the standard deviation of the excess

returns. (4) Compute the Sharpe Ratios. (5) Report the Sharpe Ratios and the average Sharpe Ratio for each year. Each step comes with Python code that we can copy and run on our machines.

However, we encountered some issues when executing the code provided by ChatGPT-4o. For instance, the excess returns were annualized, but the volatilities were not. Also, Python could not find the "date" column in the Fama-French dataset because the column name is actually "Date." Finally, the initial commands suggested by ChatGPT-4o only displayed the first five rows of the results. We resolved these issues by annualizing the stock return volatilities, correcting the column name, and adjusting the code to display all results. After these adjustments, we ran the corrected code in Python and obtained results that were very similar to those from Stata, as shown in Panel B.⁵

Overall, ChatGPT-4o demonstrates a solid understanding of the process involved in calculating the Sharpe Ratio and provides useful code for this purpose. However, users must be diligent in verifying and adjusting the provided code to ensure accuracy, particularly in handling data specifics such as column names and understanding annualization methods.

[Table 6 here]

Next, we consider another commonly used risk measure for stocks, market beta, which is not suggested in ChatGPT-4o's response in the zero-shot prompting, as shown in Table 2. Computing market beta involves multiple steps that can effectively test the computational abilities of ChatGPT-4o. To obtain the market beta, we first need to compute the daily excess return (return minus risk-free rate). Then, we run a regression by firm and year to store the slope coefficients. In Stata, this requires using local macros and loops to get the results.

⁵The small difference is due to the method of annualization. The code in Stata uses the actual number of trading days to compute the annual excess return and its annual standard deviation (e.g., 252 days in 2019 and 253 days in 2020), whereas the code suggested by ChatGPT-4o uses 252 days for all years. If the actual trading days are used for annualization in both cases, the results from Stata and Python would be identical.

After uploading the "Dow 30 Daily Returns" file, illustrated in Panel B of Table 1, and the "FamaFrench 3Factors" file, illustrated in Panel C of Table 1, we command ChatGPT-4o: "Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS, and the daily risk-free rate, pulled from Kenneth R. French - Data Library. Compute the CAPM's Beta for each stock each year."

ChatGPT-4o has an issue with inspecting the data columns, so it does not provide the results directly. Instead, ChatGPT-4o provides a detailed guide with Python code on how to analyze the data to compute the CAPM's Beta for each stock each year on the local machine.

Unfortunately, in the initial trial, we cannot obtain any results using the provided Python code because it has a couple of errors. First of all, the name of the variable "excess market return" should be 'MktRF', not 'Mkt-RF'. Second, the beta value for DOW in 2019 is missing because the value of the variable 'ret' is null on April 2, 2019. After correcting these errors in the Python code, we obtain the same results as those generated from Stata.

Overall, ChatGPT-4o demonstrates a solid understanding of the process involved in calculating market beta and provides useful code for this purpose. However, it has issues in reading the variable name precisely. Users must be diligent in verifying and adjusting the provided code to ensure accuracy, particularly in handling data specifics such as column names and dealing with missing values.

[Table 7 here]

8. ARMA-GARCH Estimation

Lastly, to capture the volatility clustering and leverage effects in the market returns, we perform an autoregressive moving average (ARMA) model with generalized autoregressive

conditional heteroskedasticity (GARCH) processes, namely ARMA–GARCH model, on the market return (vwretd). In Table 2, the response from the zero-shot prompting suggests that we should also model the return dynamics, explicitly mentioning GARCH. Estimating the ARMA-GARCH model is a complex task that involves multiple steps.

First, our data is daily, consisting of a panel of 30 firms over five years, meaning the vwretd values are repeated 30 times in the data. We need to keep only one set of the vwretd values over the 5 years (1258 trading days) and delete duplicated ones. Second, the estimation of the ARMA-GARCH model parameters is sensitive to option specifications. We obtain the results in Panel A of Table 8 in Stata using the following syntax: *arch vwretd, arch(1) garch(1) ar(1) ma(1)*.

We upload the "Dow 30 Daily Returns" file and command ChatGPT-4o: "Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS. Estimate an ARMA-GARCH model on the value-weighted return (vwretd)."

ChatGPT-4o again has an issue with executing the model fitting directly. Instead, it provides the steps (load the data, estimate ARMA model, and estimate GARCH model) with Python codes. We copy the Python codes and run them on a local machine, but they come with error messages. We then copy and paste the error message into ChatGPT-4o, which provides another set of codes. However, it still has issues to work immediately.

Specifically, in the first trial, we didn't obtain any results using the Python code provided by ChatGPT-4o because the 'ARMA' function in the 'statsmodels.tsa.api' module has been replaced by the 'ARIMA' function. Aftering correcting this error, wo obtain some estimation results, but these results are incorrect since they are produced using duplicated data. As mentioned above, the 'vwretd' values are repeated 30 times in the data. Thus, we need to delete duplicate data before

fitting a ARMA-GARCH model. The final results are shown in Panel C of Table 8. However, they are still different from those produced by Stata. We find that this is because ARMA and GARCH models are estimated simultaneously in Stata, while they are estimated sequentially in Python. Unfortunately, there is currently no package in Python that can estimate these two models jointly.

Overall, ChatGPT-4o demonstrates a fundamental understanding of the process involved in estimating ARMA-GARCH models and provides somewhat helpful code. However, users must be diligent in verifying and adjusting the provided code to ensure accuracy, particularly in handling deprecated functions and avoiding data duplication.

[Table 8 here]

9. Conclusion

OpenAI's ChatGPT-4o makes an ambitious leap forward in artificial intelligence. In this paper, we evaluate ChatGPT-4o's performance in financial data analysis, using daily stock data from CRSP for 30 companies in the Dow Jones Industrial Average and the market return and risk-free data from Ken French's library. We conduct tests including zero-shot prompting, time series analysis, risk and return analysis, and ARMA-GARCH estimation. Our findings reveal that ChatGPT-4o's performance is comparable to traditional statistical software, such as Stata, with minor discrepancies due to implementation differences. ChatGPT-4o demonstrates a robust understanding of financial data analysis, accurately interpreting complex datasets and providing insightful analyses. These capabilities may significantly enhance investment strategies and risk management practices.

Through detailed case studies and comparative analyses, we illustrate how ChatGPT-4o can transform financial research methodologies and enhance the efficiency of data-driven

decision-making processes. Despite some initial challenges with data handling and coding errors, our adjustments show that ChatGPT-4o can be a powerful tool when used with appropriate verification and validation procedures.

Our research underscores the practical utility of ChatGPT-4o in real-world financial analysis and sets the stage for its broader adoption in the finance industry. The model's advanced natural language processing capabilities, coupled with its analytical strength, offer significant benefits for financial analysts, researchers, and practitioners. These results suggest that integrating ChatGPT-4o into financial research and practice can lead to more efficient data processing, improved analytical capabilities, and better-informed investment decisions.

Looking ahead, we aim to further explore ChatGPT-4o's adaptability to different data types and contexts, including its application to other financial instruments and markets. Future research will focus on enhancing the model's performance, addressing any remaining discrepancies, and expanding its innovative applications in financial data analysis. This work lays the groundwork for ongoing advancements in AI-driven financial research, paving the way for more accurate models, improved decision-making processes, and a deeper understanding of market dynamics.

References

- Aldridge, I. (2023). The AI revolution: From linear regression to ChatGPT and beyond and how it all connects to finance. *The Journal of Portfolio Management*. DOI: 10.3905/jpm.2023.1.519.
- Ali, H. & Aysan, A. F. (2023). What will ChatGPT revolutionize in the financial industry? *Modern Finance*, 1(1), 116-129. <https://doi.org/10.61351/mf.v1i1.67>.
- Alkaissi, H., & McFarlane, S. I. (2023). Artificial hallucinations in chatgpt: Implications in scientific writing. *Cureus*, 15(2).
- Alshater, M. M. (2022). Exploring the role of artificial intelligence in enhancing academic performance: A case study of ChatGPT. *Available at SSRN 4312358*.
- Anders, B. A. (2023). Is using ChatGPT cheating, plagiarism, or both, neither, or forward thinking? *Patterns*.
- Bang, Y., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., ..., & Fung, P. (2023). A multitasking, multilingual, multimodal evaluation of chatbot on reasoning, hallucination, and interactivity. *arXiv preprint arXiv:2302.04023*.
- Bhatia, G., Nagoudi, E. M., Cavusoglu, H., & Abdul-Mageed, M. (2024). FinTral: A family of GPT-4 level multimodal financial large language models. *ArXiv*, abs/2402.10986.
- Borji, A. (2023). A categorical archive of ChatGPT failures. *arXiv preprint arXiv:2302.03494*.
- Cao, Y. & Zhai, J. (2023). Bridging the gap – the impact of ChatGPT on financial research. *Journal of Chinese Economic and Business Studies*, 21(2), 177-191.
- Chen, B., Wu, Z., & Zhao, R. (2023). From fiction to fact: the growing role of generative AI in business and finance. *Journal of Chinese Economic and Business Studies*, 21(4), 471-496.
- da Silva, J. A. T. (2023). Is ChatGPT a valid author? *Nurse Education in Practice*, 103600.
- Dai, H., Liu, Z., Liao, W., Huang, X., Wu, Z., Zhao, L., ..., & Li, X. (2023). ChatAug: Leveraging ChatGPT for text data augmentation. *arXiv preprint arXiv:2302.13007*.
- Dale, R. (2021). GPT-3: What's it good for? *Natural Language Engineering*, 27(1), 113-118.
- Dowling, M., & Lucey, B. (2023). ChatGPT for (finance) research: The Bananarama conjecture. *Finance Research Letters*, 53: 103662.
- Fahad, S. A., Salloum, S. A., Shaalan, K. (2024). The role of ChatGpt in knowledge sharing and collaboration within digital workplaces: A systematic review. In: Al-Marzouqi, A., Salloum, S. A., Al-Saidat, M., Aburayya, A., Gupta, B. (eds). *Artificial Intelligence in education: The*

- power and dangers of ChatGPT in the classroom. *Studies in big data*, 144. Springer, Cham. https://doi.org/10.1007/978-3-031-52280-2_17.
- Fatouros, G., Soldatos, J., Kouroumalis, K., Makridakis, G., & Kyriazis, D. (2023). Transforming sentiment analysis in the financial domain with ChatGPT. *arXiv:2308.07935v1*.
- Frieder, S., Pinchetti, L., Griffiths, R., Salvatori, T., Lukasiewicz, T., Petersen, P., Berner, J. (2023). Mathematical capabilities of ChatGPT. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds). *Advances in neural information processing systems*, 36, 27699-27744.
- Frye, B. L. (2022). Should using an AI text generator to produce academic writing be plagiarism? *Fordham Intellectual Property, Media & Entertainment Law Journal*, Forthcoming.
- Gao, C. A., Howard, F. M., Markov, N. S., Dyer, E. C., Ramesh, S., Luo, Y., & Pearson, A. T. (2022). Comparing scientific abstracts generated by ChatGPT to original abstracts using an artificial intelligence output detector, plagiarism detector, and blinded human reviewers. *bioRxiv*, 2022-12.
- González-Padilla, D. A. (2022). Concerns about the potential risks of artificial intelligence in manuscript writing. *Journal of Urology*, 10-1097.
- Hosseini, M., Rasmussen, L. M., & Resnik, D. B. (2023). Using AI to write scholarly publications. *Accountability in Research*, 1-9.
- Jha, M., Qian, J., Weber, M., & Yang, B. (2023). ChatGPT and corporate policies. Working paper. *Available at SSRN 4521096*.
- Khalil, M. & Er, E. (2023). Will ChatGPT get you caught? Rethinking of plagiarism detection. *arXiv preprint arXiv:2302.04335*.
- Khan, M. S. & Umer, H. (2024). ChatGPT in finance: Applications, challenges, and solutions. *Heliyon*, 10(2), e24890.
- Kim, A. G., Muhn, M., & Nikoleav, V. V. (2024). Financial statement analysis with large language models. Chicago Booth Research Paper Forthcoming, Fama-Miller Working Paper, <https://ssrn.com/abstract=4835311>.
- Korinek, A. (2023). Language models and cognitive automation for economic research. NBER working paper series. <https://www.nber.org/papers/w30957>.
- Li, X., Feng, H., Yang, H., & Huang, J. (2024). Can ChatGPT reduce human financial analysts' optimistic biases? *Economic and Political Studies*, 12(1), 20-33.

- Liebrecht, M., Schleifer, R., Buadze, A., Bhugra, D., & Smith, A. (2023). Generating scholarly content with ChatGPT: Ethical challenges for medical publishing. *The Lancet Digital Health*, 5(3), e105-e106.
- Lin, Z. (2023). Why and how to embrace AI such as ChatGPT in your academic life. *Royal Society Open Science*, 10(8): 230658.
- Liu, X., Zheng, Y., Du, Z., Ding, M., Qian, Y., Yang, Z., & Tang, J. (2023). GPT understands, too. *AI Open*, 62. <https://doi.org/10.1016/j.aiopen.2023.08.012>
- Lopez-Lira, A. & Tang, Y. (2024). Can ChatGPT forecast stock price movements? Return predictability and large language models. Working paper. Available at SSRN 4412788.
- Lund, B. D., & Wang, T. (2023). Chatting about ChatGPT: How may AI and GPT impact academia and libraries? *Library Hi Tech News*.
- Pelster, M. & Val, J. (2024). Can ChatGPT assist in picking stocks? *Finance Research Letters*. 59: 104786.
- Rahman, M. M. & Watanobe, Y. (2023). ChatGPT for education and research: Opportunities, threats, and strategies. *Applied Sciences*. 13, 5783. <https://doi.org/10.3390/app13095783>
- Rice, S., Crouse, S. R., Winter, S. R., & Rice, C. (2024). The advantages and limitations of using ChatGPT to enhance technological research. *Technology in Society*, 76, <https://doi.org/10.1016/j.techsoc.2023.102426>.
- Shen, Y., Heacock, L., Elias, J., Hentel, K. D., Reig, B., Shih, G., & Moy, L. (2023). ChatGPT and other large language models are double-edged swords. *Radiology*, 230163.
- Shue, E., Liu, L., Li, B., Feng, Z., Li, X., & Hu, G. (2023). Empowering beginners in bioinformatics with ChatGPT. *Quantitative Biology*, 11(2), 105-108.
- Smales, L. A. (2023). Classification of RBA monetary policy announcements using ChatGPT. *Finance Research Letters*, 58(C): 104514.
- Stokel-Walker, C. (2023). ChatGPT listed as author on research papers: many scientists disapprove. *Nature*.
- Thorp, H. H. (2023). ChatGPT is fun, but not an author. *Science*, 379(6630), 313.
- van Dis, E. A., Bollen, J., Zuidema, W., van Rooij, R., & Bockting, C. L. (2023). ChatGPT: Five priorities for research. *Nature*, 614(7947), 224-226.
- Wang, J., Ye, Q., Liu, L., Guo, N.L., & Hu, G. (2024). Scientific figures interpreted by ChatGPT: strengths in plot recognition and limits in color perception. *NPJ Precision Oncology*, 8.

- Yang, Z., Li, L., Lin, K., Wang, J., Lin, C., Liu, Z., & Wang, L. (2023). The dawn of LMMs: preliminary explorations with GPT-4V(ision). ArXiv, abs/2309.17421.
- Yeo-Teh, N. S. L., & Tang, B. L. (2023). Letter to Editor: NLP systems such as ChatGPT cannot be listed as an author because these cannot fulfill widely adopted authorship criteria. *Accountability in research*, Feb 13, 1-3.
- Yue, T., Au, D., Au, C. C., & Iu, K. Y. (2023). Democratizing financial knowledge with ChatGPT by OpenAI: Unleashing the power of technology. *Available at SSRN 4346152*.
- Zaremba, A. & Demir, E. (2023). ChatGPT: Unlocking the future of NLP in finance. *Modern Finance*, 1(1), 93-98.
- Zhu, G., Fan, X., Hou, C., Zhong, T., Seow, P., Shen-Hsing, A C., ... & Poh, T. L. (2023). Embrace opportunities and face challenges: Using ChatGPT in undergraduate students' collaborative interdisciplinary learning. *Computer Science*. arXiv:2305.18616.

Table 1: Data

This table reports the structure of the data that we use in the empirical tests. Panel A is 30 components from the Dow Jones Industrial Average Stocks List. Panel B is the data structure for the "Dow 30 Daily Returns" File. Panel C is the data structure for the "FamaFrench_3Factors" file.

Panel A: Dow Jones Industrial Average Stocks List

SYMBOL	NAME
AAPL	Apple Inc
AMGN	Amgen Inc
AMZN	Amazon.com Inc
AXP	American Express Co
BA	Boeing Co
CAT	Caterpillar Inc
CRM	Salesforce Inc
CSCO	Cisco Systems Inc
CVX	Chevron Corp
DIS	Walt Disney Co
DOW	Dow Inc
GS	Goldman Sachs Group Inc
HD	Home Depot Inc
HON	Honeywell International Inc
IBM	International Business Machines Corp
INTC	Intel Corp
JNJ	Johnson & Johnson
JPM	JPMorgan Chase & Co
KO	Coca-Cola Co
MCD	McDonald's Corp
MMM	3M Co
MRK	Merck & Co Inc
MSFT	Microsoft Corp
NKE	Nike Inc
PG	Procter & Gamble Co
TRV	Travelers Companies Inc
UNH	Unitedhealth Group Inc
V	Visa Inc
VZ	Verizon Communications Inc
WMT	Walmart Inc

Panel B: Data Structure for the "Dow 30 Daily Returns" File

PERMNO	date	TICKER	COMNAM	PERMCO	CUSIP	VOL	RET	BID	ASK	SHROUT	NUMTRD	RETX	vwretd	sprtrn
10107	1/2/2019	MSFT	MICROSOFT CORP	8048	59491810	35347045	-0.00443	101.12	101.13	7683000	239618	-0.00443	0.001796	0.001269
10107	1/3/2019	MSFT	MICROSOFT CORP	8048	59491810	42570779	-0.03679	97.35	97.38	7683000	302446	-0.03679	-0.02104	-0.02476
10107	1/4/2019	MSFT	MICROSOFT CORP	8048	59491810	44032862	0.046509	101.93	101.94	7683000	301838	0.046509	0.03341	0.034336
10107	1/7/2019	MSFT	MICROSOFT CORP	8048	59491810	35650303	0.001275	102.04	102.06	7683000	240581	0.001275	0.009202	0.00701
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
92655	12/26/2023	UNH	UNITEDHEALTH GROUP INC	7267	91324P10	1390912	-0.000538	520.10999	520.12	924925		-0.000538	0.005218	0.004232
92655	12/27/2023	UNH	UNITEDHEALTH GROUP INC	7267	91324P10	1851840	0.005307	522.95001	523.09003	924925		0.005307	0.001995	0.00143
92655	12/28/2023	UNH	UNITEDHEALTH GROUP INC	7267	91324P10	2001208	0.004036	525.07001	525.21002	924925		0.004036	-0.000108	0.00037
92655	12/29/2023	UNH	UNITEDHEALTH GROUP INC	7267	91324P10	2080197	0.002991	526.53998	526.94	924925		0.002991	-0.004045	-0.002826

Panel C: Data Structure for the "FamaFrench_3Factors" File

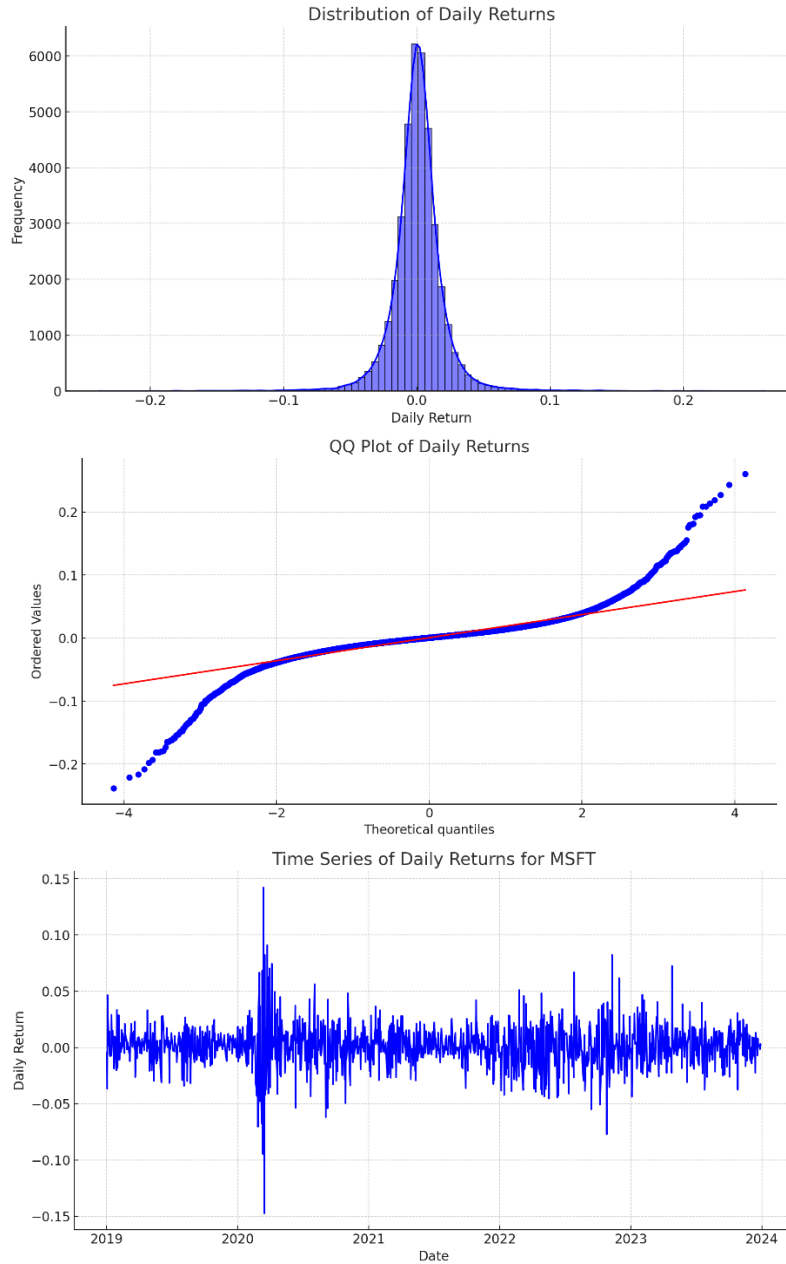
Date	MktRF	SMB	HML	RF
20190102	0.0023	0.0059	0.0111	0.0001
20190103	-0.0245	0.0036	0.012	0.0001
20190104	0.0355	0.0041	-0.007	0.0001
20190107	0.0094	0.01	-0.0075	0.0001
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
20231226	0.0048	0.0069	0.0046	0.00021
20231227	0.0016	0.0014	0.0012	0.00021
20231228	-0.0001	-0.0036	0.0002	0.00021
20231229	-0.0043	-0.0113	-0.0037	0.00021

Table 2: Zero-Shot Prompting

This table reports the financial analysis with a 0-shot prompt. Panel A is the results generated using ChatGPT-4o. The "Prompt" refers to the request made by the authors, while the "Response" refers to the feedback provided by ChatGPT-4o). Panel B is an "Assessment" of the quality and accuracy of the ChatGPT-4o response provided by the authors.

Panel A: Results from ChatGPT-4o

Prompts:
[upload from computer the "Dow 30 Daily Returns" file] Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS. Perform analysis that can be used in academic papers. Illustrate the meaning and implications of the results.
Response:
The dataset contains daily returns data for the Dow 30 components from 2019 to 2023. Here are the key columns in the dataset: • PERMNO: Permanent number assigned to each security. • date: Date of the observation. • TICKER: Stock ticker symbol. • COMNAM: Company name. • PERMCO: Permanent company number. • CUSIP: Committee on Uniform Securities Identification Procedures number. • VOL: Trading volume. • RET: Daily return. • BID: Bid price. • ASK: Ask price. • SHROUT: Number of shares outstanding. • NUMTRD: Number of trades. • RETX: Return excluding dividends. • vwretd: Value-weighted return. • sprtrn: Spread return. Statistical Analysis 1. Descriptive Statistics: We will calculate basic descriptive statistics (mean, median, standard deviation, etc.) for daily returns. 2. Distribution Analysis: We will examine the distribution of daily returns, including skewness and kurtosis, and visualize it using histograms and QQ plots. 3. Time Series Analysis: We will analyze the time series properties of the returns, including autocorrelation and volatility clustering. 4. Risk and Return Analysis: We will compute key metrics such as annualized returns, annualized volatility, Sharpe ratio, and Beta. 5. Correlation Analysis: We will analyze the correlation between the returns of different Dow 30 components. Implementation Let's start with the descriptive statistics and distribution analysis. [View Analysis]



Descriptive Statistics

Here are the basic descriptive statistics for the daily returns (RET):

- **Count:** Number of observations
- **Mean:** Average daily return
- **Std:** Standard deviation of daily returns
- **Min:** Minimum daily return
- **25%:** 25th percentile
- **50%:** Median (50th percentile)
- **75%:** 75th percentile
- **Max:** Maximum daily return
- **Skewness:** Measure of asymmetry in the distribution of returns
- **Kurtosis:** Measure of the "tailedness" of the distribution

Distribution Analysis

Histogram and KDE Plot

The histogram and kernel density estimate (KDE) plot show the distribution of daily returns. The distribution appears to be approximately normal but with noticeable tails, indicating the presence of extreme values or outliers.

QQ Plot

The QQ plot compares the distribution of daily returns to a standard normal distribution. Deviations from the line suggest departures from normality. The tails of the distribution show significant deviations, indicating the presence of outliers or heavy tails.

Time Series Analysis

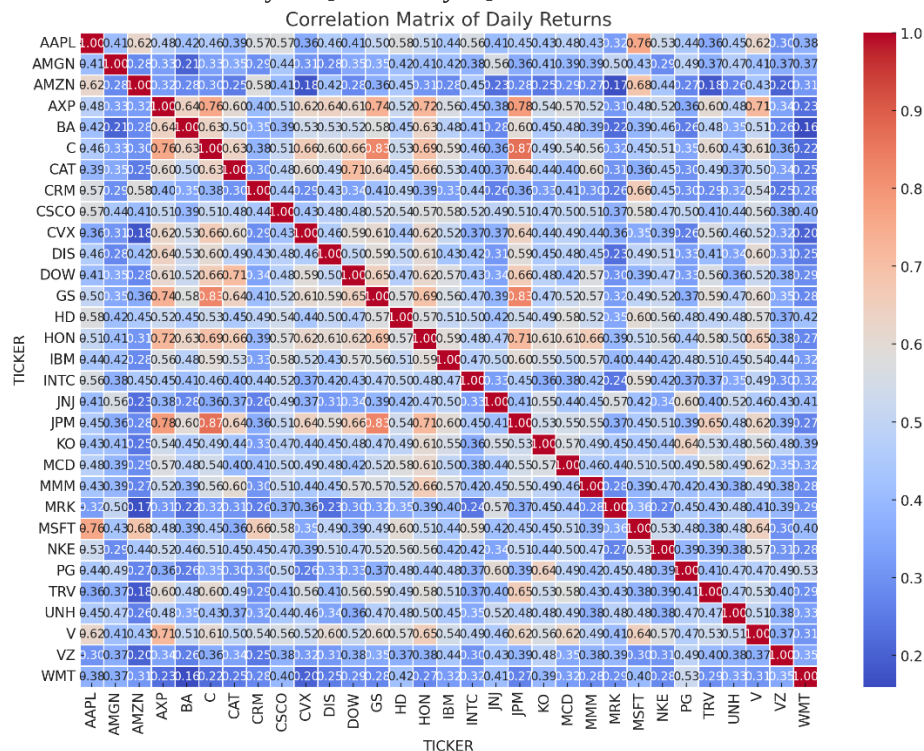
Time Series Plot for MSFT

The time series plot for Microsoft's daily returns highlights the fluctuations over time. Notable patterns include periods of high volatility and clustering of extreme values.

Implications

1. **Risk and Return:** The mean daily return provides insight into the expected performance, while the standard deviation and kurtosis highlight the risk and potential for extreme returns.
2. **Distribution Properties:** The skewness and kurtosis indicate that the distribution of returns is not perfectly normal, suggesting that models assuming normality may underestimate the risk of extreme events.
3. **Volatility Clustering:** The time series plot reveals periods of increased volatility, which is typical in financial markets and has implications for risk management and portfolio optimization.
4. **Modeling Returns:** The deviations from normality suggest that more sophisticated models, such as GARCH or EVT (Extreme Value Theory), might be necessary for accurate risk assessment.

Next, we'll proceed with further analysis, including calculating annualized metrics, examining correlations, and performing additional time series analysis. [\[View Analysis\]](#)



Annualized Metrics

Here are the key metrics for each Dow 30 component:

- **Annualized Return:** The compounded return over a year.
- **Annualized Volatility:** The standard deviation of returns over a year.
- **Sharpe Ratio:** The risk-adjusted return, calculated as $\frac{\text{Annualized Return} - \text{Risk-Free Rate}}{\text{Annualized Volatility}}$

Key Observations

- **Highest Annualized Return:** AAPL (Apple) at approximately 45.89%
- **Highest Annualized Volatility:** BA (Boeing) at approximately 50.40%
- **Highest Sharpe Ratio:** AAPL (Apple) at 1.362

These metrics indicate that Apple has provided the highest risk-adjusted return over this period, making it an attractive investment despite its relatively high volatility.

Correlation Analysis

The heatmap of the correlation matrix shows the relationships between the daily returns of the Dow 30 components. Key observations include:

- **High Correlation:** Companies within the same industry or sector often exhibit higher correlations. For example, financial firms like JPM (JPMorgan Chase) and GS (Goldman Sachs) show a strong positive correlation.
- **Low Correlation:** Companies from different sectors might show lower correlations, offering diversification benefits. For instance, technology firms like MSFT (Microsoft) and healthcare companies like JNJ (Johnson & Johnson) show moderate to low correlations.

Implications

1. **Portfolio Diversification:** The correlation analysis supports the principle of diversification. By combining assets with low or negative correlations, investors can reduce portfolio risk without significantly sacrificing returns.
2. **Risk Management:** Understanding the volatility and correlation of different stocks helps in constructing portfolios that align with the investor's risk tolerance and investment objectives.
3. **Investment Strategy:** Stocks with high Sharpe ratios, like AAPL, indicate superior risk-adjusted returns, making them attractive for investors seeking to maximize returns for a given level of risk.

Conclusion

The analysis of the Dow 30 components from 2019 to 2023 reveals important insights into their risk-return profiles and interrelationships. This information is crucial for academic research, investment strategies, and risk management practices. Further studies could involve more sophisticated modeling techniques to better understand market dynamics and predict future performance. [View Analysis]

Panel B: Assessment of the Results from ChatGPT-4o

1. In identifying the variables/column headers of the input data, out of 15 columns, ChatGPT-4o correctly identifies the header names for 14 of them. The misidentified column is *sprtrn*, which should represent the S&P 500 Composite Index Return but is labeled as Spread Return in the responses.
2. No actual numbers are provided in the descriptive analysis, but it does include the names of those common descriptive statistics that we should have in a financial paper.
3. ChatGPT-4o understands this is time series data and suggests relevant time series analyses, such as risk and return relationships and modeling returns.

Table 3: Summary Statistics

This table reports the summary statistics for the daily returns data of Dow 30 components from 2019 to 2023. Panel A presents the results generated using Stata. Panel B shows the results generated using ChatGPT-4o. The "Prompt" refers to the request made by the authors, while the "Response" refers to the feedback provided by ChatGPT-4o. Panel C offers an "Assessment" of the quality and accuracy of the ChatGPT-4o response provided by the authors.

Panel A: Results from Stata

Variable	Mean	Median	Standard Deviation	Minimum	Maximum	Number of Observations
ret	0.000616	0.000671	0.019557	-0.238484	0.260449	38935
vol	11,965,600	6,329,270	16,900,200	416,678	331,573,000	38936
bid	213.455	147.610	389.842	21.990	3,731.410	38936
ask	213.510	147.625	390.018	22.000	3,733.390	38936
shrout	2,326,370	1,274,960	2,756,090	228,399	17,102,500	38936
vwret	0.000605	0.000987	0.013280	-0.118168	0.091556	38936

Panel B: Results from ChatGPT-4o

Prompts:

[upload from computer the "Dow 30 Daily Returns" file]

Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS. Provide a summary statistic table that includes the mean, median, standard deviation, minimal, maximum, and number of observations for the following variables: return (ret), volume (vol), bid price (bid), ask price (ask), shares outstanding (shrout), and value-weighted return (vwret).

Response:

Here are the summary statistics for the daily returns data of Dow 30 components from 2019 to 2023:

Variable	Mean	Median	Standard Deviation	Minimum	Maximum	Number of Observations
RET	0.000409	0.000548	0.018073	-0.308225	0.350837	38936
VOL	11,965,626	6,329,272	16,900,193	416,678	331,572,902	38936
BID	213.455	147.610	389.842	21.990	3731.410	38936
ASK	213.510	147.625	390.018	22.000	3733.390	38936
SHROUT	2,326,367	1,274,956	2,756,085	228,399	17,102,540	38936
vwret	0.000605	0.000987	0.013280	-0.118168	0.091556	38936

These statistics provide a comprehensive overview of the dataset, highlighting key measures of central tendency and dispersion. [View Analysis]

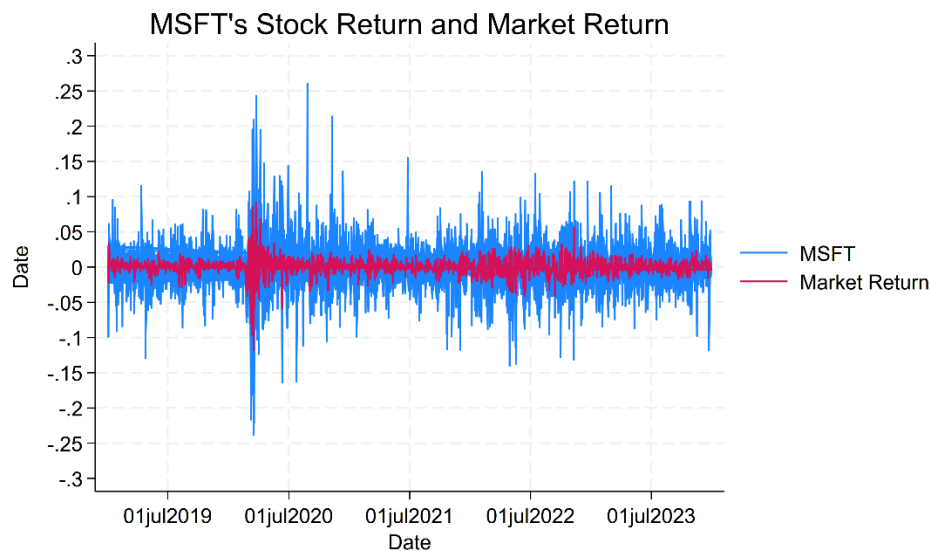
Panel C: Assessment of the Results from ChatGPT-4o

The statistics for return (ret) and volume (vol) do not match those generated by Stata, while those for bid price (bid), ask price (ask), shares outstanding (shrout), and value-weighted return (vwret) do match. This assessment indicates that ChatGPT-4o can process and analyze most of the data accurately, but discrepancies in some variables highlight the need for further validation and cross-checking with traditional statistical software.

Table 4: Time Series Analysis of Returns of Microsoft and Market

This table evaluates a time series plot for Microsoft's (ticker = MSFT) return (ret) and the market's (vwretd) return from 2019 to 2023. Panel A presents the results generated using Stata. Panel B shows the results generated using ChatGPT-4o. The "Prompt" refers to the request made by the authors, while the "Response" refers to the feedback provided by ChatGPT-4o. Panel C offers an "Assessment" of the quality and accuracy of the ChatGPT-4o response provided by the authors.

Panel A: Results from Stata



Panel B: Results from ChatGPT-4o

Prompts:
[upload from computer the "Dow 30 Daily Returns" file] Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS. Provide a time series plot for the return (ret) of Microsoft (ticker = MSFT) and the market return (vwretd).
Response:
It seems there's an issue with generating the plot directly. Let's save the plot as an image file and provide it that way. I'll try creating the plot again and save it as an image file. [View Analysis] It looks like we're facing persistent issues with generating the plot. Let's try an alternative approach by summarizing the data trends instead. For the analysis of the daily returns of Microsoft (RET) and the market return (vwretd) from 2019 to 2023: Microsoft (RET) • Trend: Over the observed period, Microsoft's daily returns have shown typical fluctuations associated with a high-profile technology company. This includes notable peaks and troughs corresponding to major market events and company-specific news. • Volatility: The daily return volatility reflects the underlying market conditions, including periods of heightened uncertainty like the COVID-19 pandemic. Market Return (vwretd)

- **Trend:** The value-weighted market return similarly shows fluctuations, but these are generally more subdued compared to individual stock returns. This is because it represents a broader market index, which is inherently more stable.
- **Volatility:** Market returns exhibit less volatility on a day-to-day basis compared to individual stocks due to the diversification effect.

Correlation

- **Positive Correlation:** Both Microsoft's returns and the market returns tend to move in the same direction, showing a positive correlation typical for a large-cap stock within a major index.

For a more detailed visual representation, you can use statistical software or tools like Python, R, or Excel to plot the data directly on your local machine. If you need further assistance with the steps to create the plot locally, please let me know! [[View Analysis](#)]

Panel C: Assessment of the Results from ChatGPT-4o

ChatGPT-4o has difficulty generating the figure from the data based on the prompt. However, Table 2: Zero-Shot Prompting does demonstrate the ability of ChatGPT-4o to visualize the time series plots. This indicates some instability in ChatGPT-4o's ability to generate responses consistently.

Table 5: Returns Around COVID-19 Lockdown

This table assesses the stock returns on the trading days before and after the first COVID-19 lockdown day (Sunday, March 15, 2020). Panel A presents the results generated using Stata. Panel B shows the results generated using ChatGPT-4o. The "Prompt" refers to the request made by the authors, while the "Response" refers to the feedback provided by ChatGPT-4o. Panel C offers an "Assessment" of the quality and accuracy of the ChatGPT-4o response provided by the authors.

Panel A: Results from Stata

TICKER	13-Mar-2020	16-Mar-2020	Total
AAPL	0.119808	-0.128647	-0.008839
AMGN	0.108977	-0.066403	0.042574
AMZN	0.064648	-0.053697	0.010951
AXP	0.192386	-0.137851	0.054535
BA	0.099199	-0.238484	-0.139285
C	0.179843	-0.192986	-0.013143
CAT	0.079991	-0.062525	0.017466
CRM	0.051142	-0.158885	-0.107743
CSCO	0.133735	-0.104410	0.029325
CVX	0.093889	-0.164469	-0.070580
DIS	0.116654	-0.073254	0.043400
DOW	0.085340	-0.143636	-0.058296
GS	0.175803	-0.127053	0.048750
HD	0.079123	-0.197938	-0.118815
HON	0.107972	-0.094639	0.013333
IBM	0.049995	-0.082168	-0.032173
INTC	0.195213	-0.180415	0.014798
JNJ	0.070808	-0.053317	0.017491
JPM	0.180125	-0.149649	0.030476
KO	0.036472	-0.066227	-0.029755
MCD	0.041145	-0.158754	-0.117609
MMM	0.060638	-0.076016	-0.015378
MRK	0.039087	-0.088990	-0.049903
MSFT	0.142169	-0.147390	-0.005221
NKE	0.018598	-0.116301	-0.097703
PG	0.120090	-0.048830	0.071260
TRV	0.051576	-0.208004	-0.156428
UNH	0.090691	-0.172769	-0.082078
V	0.098388	-0.135472	-0.037084
VZ	0.058008	-0.058704	-0.000696
WMT	0.096588	-0.064330	0.032258
Total	0.098003	-0.121039	-0.023036

Panel B: Results from ChatGPT-4o

Prompts:

[upload from computer the "Dow 30 Daily Returns" file]

Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS.

Provide the return (ret) of each stock on the trading days before and after the first COVID lockdown day (Sunday, March 15, 2020). Compute the total return of each stock's two trading days, and the average return of all stocks' two trading days.

Response:

It seems that there is an issue with accessing or interpreting the data. Let's re-attempt by ensuring we handle the file and its columns correctly. I'll reload the file and ensure proper column handling.

Let's try this step-by-step again to pinpoint the issue. [View Analysis]

It appears that there is a persistent issue with accessing the data. Let's approach this step-by-step:

1. **Check if the data is loaded correctly.**
2. **Ensure that the date conversion and column selection are correct.**
3. **Isolate the trading days before and after the specified lockdown date.**

Let's retry these steps carefully. [View Analysis]

It seems there's an ongoing issue with loading the CSV file. To proceed effectively, I'll outline the steps you can follow to analyze the data on your local machine using Python:

1. **Load the Data:** Ensure the CSV file is loaded correctly using **pandas**.

Python	Copy code
import pandas as pd	
# Load the data	
file_path = 'path/to/your/Dow 30 Daily Returns.csv'	
data = pd.read_csv(file_path)	

2. **Convert Date Column:** Convert the **date** column to **datetime** format.

Python	Copy code
data['date'] = pd.to_datetime(data['date'])	

3. **Identify Trading Days:** Identify the trading days before and after March 15, 2020.

Python	Copy code
lockdown_date = pd.to_datetime("2020-03-15")	
trading_day_before = data[data['date'] < lockdown_date]['date'].max()	
trading_day_after = data[data['date'] > lockdown_date]['date'].min()	

4. **Filter Data:** Filter the data for these trading days.

Python	Copy code
data_before_after = data[(data['date'] == trading_day_before) (data['date'] == trading_day_after)]	

5. **Calculate Returns:** Calculate the total return of each stock over these two days and the average return.

Python	Copy code
total_returns = data_before_after.groupby("TICKER")["RET"].sum()	
average_return = total_returns.mean()	
print(total_returns)	
print(f'Average Return: {average_return}')	

If you follow these steps on your local machine, you should be able to analyze the daily returns data of the Dow 30 components effectively. If you encounter any specific errors or issues during these steps, please let me know, and I can assist you further! [View Analysis]

Panel C: Assessment of the Results from ChatGPT-4o

We used the above Python code suggested by ChatGPT-4o and obtained the following results.

TICKER	Total
AAPL	-0.008839
AMGN	0.042574
AMZN	0.010951
AXP	0.054535
BA	-0.139285
C	-0.013143
CAT	0.017466
CRM	-0.107743
CSCO	0.029325
CVX	-0.070580
DIS	0.043400
DOW	-0.058296
GS	0.048750
HD	-0.118815
HON	0.013333
IBM	-0.032173
INTC	0.014798
JNJ	0.017491
JPM	0.030476
KO	-0.029755
MCD	-0.117609
MMM	-0.015378
MRK	-0.049903
MSFT	-0.005221
NKE	-0.097703
PG	0.071260
TRV	-0.156428
UNH	-0.082078
V	-0.037084
VZ	-0.000696
WMT	0.032258
Total	-0.023036

Thus, the Python code given by ChatGPT-4o is correct but incomplete. It doesn't display the daily returns on March 13 and March 16. To fix the problem, we add a couple of lines of code to the initial one. After running the corrected code shown below, we obtain the same results as those in Panel A generated by Stata.

```
import pandas as pd

# Load the data
```

```

file_path = 'path/to/your/Dow 30 Daily Returns.csv'
data = pd.read_csv(file_path)

# Convert date Column
data['date'] = pd.to_datetime(data['date'])

# Identify trading days
lockdown_date = pd.to_datetime('2020-03-15')
trading_day_before = data[data['date'] < lockdown_date]['date'].max()
trading_day_after = data[data['date'] > lockdown_date]['date'].min()

# Filter data
data_before_after = data[(data['date'] == trading_day_before) | (data['date'] == trading_day_after)]
data_before_after = data_before_after[['TICKER', 'date', 'RET']]
data_before_after = data_before_after.sort_values(by=['TICKER', 'date'])
pd.set_option('display.max_columns', None)
print(data_before_after.to_string())

# Calculate total returns
total_returns = data_before_after.groupby('TICKER')['RET'].sum()
total_perday_returns = data_before_after.groupby('date')['RET'].mean()
average_return = total_returns.mean()

# Display the results
print(total_returns)
print(total_perday_returns)
print(f'Average Return: {average_return}')

```

TICKER	13-Mar-2020	16-Mar-2020	Total
AAPL	0.119808	-0.128647	-0.008839
AMGN	0.108977	-0.066403	0.042574
AMZN	0.064648	-0.053697	0.010951
AXP	0.192386	-0.137851	0.054535
BA	0.099199	-0.238484	-0.139285
C	0.179843	-0.192986	-0.013143
CAT	0.079991	-0.062525	0.017466
CRM	0.051142	-0.158885	-0.107743
CSCO	0.133735	-0.104410	0.029325
CVX	0.093889	-0.164469	-0.070580
DIS	0.116654	-0.073254	0.043400
DOW	0.085340	-0.143636	-0.058296
GS	0.175803	-0.127053	0.048750
HD	0.079123	-0.197938	-0.118815
HON	0.107972	-0.094639	0.013333
IBM	0.049995	-0.082168	-0.032173
INTC	0.195213	-0.180415	0.014798
JNJ	0.070808	-0.053317	0.017491
JPM	0.180125	-0.149649	0.030476

KO	0.036472	-0.066227	-0.029755
MCD	0.041145	-0.158754	-0.117609
MMM	0.060638	-0.076016	-0.015378
MRK	0.039087	-0.088990	-0.049903
MSFT	0.142169	-0.147390	-0.005221
NKE	0.018598	-0.116301	-0.097703
PG	0.120090	-0.048830	0.071260
TRV	0.051576	-0.208004	-0.156428
UNH	0.090691	-0.172769	-0.082078
V	0.098388	-0.135472	-0.037084
VZ	0.058008	-0.058704	-0.000696
WMT	0.096588	-0.064330	0.032258
Total	0.098003	-0.121039	-0.023036

Table 6: Sharpe Ratio Analysis

This table assesses the Sharpe ratio, (stock return - risk-free rate) / (standard deviation of the stock return), for each stock each year. Panel A presents the results generated using Stata. Panel B shows the results generated using ChatGPT-4o. The "Prompt" refers to the request made by the authors, while the "Response" refers to the feedback provided by ChatGPT-4o. Panel C offers an "Assessment" of the quality and accuracy of the ChatGPT-4o response provided by the authors.

Panel A: Results from Stata

TICKER	2019	2020	2021	2022	2023
AAPL	3.251206	1.745468	1.380334	-0.77026	2.079339
AMGN	1.191135	-0.06218	0.043803	0.8633	0.395884
AMZN	0.891517	1.958081	0.0986	-1.00693	2.202766
AXP	1.741743	-0.02399	1.318358	-0.27628	0.8745
BA	0.039458	-0.38989	-0.16568	-0.14373	1.124431
C	2.230223	-0.29584	0.038892	-0.69407	0.527584
CAT	0.62998	0.587773	0.624957	0.505119	0.705026
CRM	0.633002	0.684419	0.504112	-1.03044	3.027711
CSCO	0.467521	-0.09327	2.384514	-0.80602	0.206838
CVX	0.697617	-0.42961	1.891078	1.712105	-0.76628
DIS	1.360394	0.504435	-0.58419	-1.20843	-0.02319
DOW	0.003003	0.122246	0.237751	-0.26553	0.424431
GS	1.585639	0.325115	1.888141	-0.31028	0.437651
HD	1.529674	0.55179	2.963595	-0.73857	0.350903
HON	1.954297	0.495626	-0.01511	0.136817	-0.25516
IBM	1.019095	-0.03861	0.71867	0.384131	1.030328
INTC	1.033142	-0.28115	0.191356	-1.23999	2.235086
JNJ	0.836069	0.343391	0.789819	0.257685	-0.78172
JPM	2.358938	-0.1082	1.297706	-0.46437	1.187297
KO	1.062758	0.058642	0.749322	0.461006	-0.66602
MCD	0.770792	0.271025	1.817212	-0.04574	0.696737
MMM	-0.25117	0.063578	0.26232	-1.12427	-0.28893
MRK	1.069891	-0.23757	0.082563	2.377886	-0.1987
MSFT	2.735991	0.954548	2.493022	-0.82381	2.034473
NKE	1.699654	0.971769	0.668222	-0.71205	-0.38724
PG	2.229552	0.42118	1.424969	-0.29119	-0.36953
TRV	0.875852	0.102008	0.643098	0.908932	-0.04441
UNH	0.690608	0.430659	2.362684	0.223695	-0.18567
V	2.236204	0.389683	-0.01329	-0.15375	1.317385
VZ	0.72356	-0.02082	-0.52791	-0.93618	-0.089
WMT	1.904296	0.724101	0.117798	-0.07003	0.488505
Total	1.264569	0.31369112	0.82860385	-0.17036265	0.55777513

Panel B: Results from ChatGPT-4o

Prompts:

[upload from computer the "Dow 30 Daily Returns" file & the "FamaFrench 3Factors" file]

Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS, and the daily risk free rate, pulled from Kenneth R. French - Data Library.

Compute the annualized excess return (raw return – risk free rate) of each stock.

Compute the standard deviation of excess returns for each stock each year.

Then, calculate the Sharpe Ratio, the risk-adjusted return, for each stock each year.

Report the Sharpe Ratio for each stock each year and the average Sharpe Ratio of all stocks each year.

Response:

It looks like there's an issue with generating the computations directly. Let's break down the process into more manageable steps to ensure we capture all necessary calculations. Here's the step-by-step approach:

1. **Load and merge the data.**
2. **Compute the annualized excess returns.**
3. **Calculate the standard deviation of the excess returns.**
4. **Compute the Sharpe Ratios.**
5. **Report the Sharpe Ratios and the average Sharpe Ratio for each year.**

I'll guide you through each of these steps. Please use the following code snippets on your local machine to achieve the required analysis.

1. Load and Merge the Data

Python

Copy code

```
import pandas as pd

# Load the Dow 30 daily returns data
dow_data_path = 'path/to/Dow 30 Daily Returns.csv'
dow_data = pd.read_csv(dow_data_path)

# Load the Fama-French 3 factors data (including the risk-free rate)
ff_data_path = 'path/to/FamaFrench_3Factors.csv'
ff_data = pd.read_csv(ff_data_path)

# Convert date columns to datetime format
dow_data['date'] = pd.to_datetime(dow_data['date'])
ff_data['date'] = pd.to_datetime(ff_data['date'], format='%Y%m%d')

# Merge the datasets on the date column
merged_data = pd.merge(dow_data, ff_data, on='date', how='inner')
```

2. Compute the Annualized Excess Returns

Python

Copy code

```
# Calculate the excess return (raw return - risk free rate)
merged_data['excess_ret'] = merged_data['RET'] - merged_data['RF']

# Extract the year from the date for annual calculations
merged_data['year'] = merged_data['date'].dt.year

# Function to annualize returns
def annualize_return(daily_returns):
    return (1 + daily_returns).prod() ** (252 / len(daily_returns)) - 1
```

<pre># Compute the annualized excess return for each stock each year annual_excess_return = merged_data.groupby(['TICKER', 'year'])['excess_ret'].apply(annualize_return).reset_index() annual_excess_return.columns = ['TICKER', 'year', 'annual_excess_return']</pre>	
3. Calculate the Standard Deviation of Excess Returns	
Python	Copy code
<pre># Compute the standard deviation of excess returns for each stock each year std_excess_return = merged_data.groupby(['TICKER', 'year'])['excess_ret'].std().reset_index() std_excess_return.columns = ['TICKER', 'year', 'std_excess_return']</pre>	
4. Compute the Sharpe Ratios	
Python	Copy code
<pre># Merge the annualized return and standard deviation data annual_stats = pd.merge(annual_excess_return, std_excess_return, on=['TICKER', 'year']) # Calculate the Sharpe Ratio (annualized excess return / standard deviation of excess return) annual_stats['sharpe_ratio'] = annual_stats['annual_excess_return'] / annual_stats['std_excess_return']</pre>	
5. Report the Sharpe Ratios and the Average Sharpe Ratio for Each Year	
Python	Copy code
<pre># Calculate the average Sharpe Ratio for all stocks each year average_sharpe_ratio = annual_stats.groupby('year')['sharpe_ratio'].mean().reset_index() average_sharpe_ratio.columns = ['year', 'average_sharpe_ratio'] # Merge the average Sharpe Ratio with the annual stats annual_stats = pd.merge(annual_stats, average_sharpe_ratio, on='year', how='left') # Display the results print(annual_stats.head()) print(average_sharpe_ratio.head())</pre>	
By following these steps, you should be able to compute the required statistics on your local machine. If you encounter any specific errors, please provide details, and I can assist you further.	

Panel C: Assessment of the Results from ChatGPT-4o

We encountered some problems when running the code provided by ChatGPT-4o. First, Python could not find the "date" column in the Fama-French dataset. By using the command "print(ff_data.columns)," we discovered that the column name should be "Date." Second, the commands in the last two lines, "print(annual_stats.head())" and "print(average_sharpe_ratio.head())," only display the first five rows of the results. Thus, we changed them to "print(annual_stats.to_string())" and "print(average_sharpe_ratio.to_string())." After these adjustments, we obtained the following results.

TICKER	2019	2020	2021	2022	2023
AAPL	51.6112985	27.6178279	21.9121292	-12.244374	33.1895478
AMGN	18.9086843	-0.9851038	0.69535698	13.7365684	6.31156242
AMZN	14.1523959	30.9843846	1.56523015	-15.996577	35.1909812
AXP	27.6493114	-0.3801246	20.9282788	-4.3936676	13.9494736
BA	0.62638058	-6.1817707	-2.6300849	-2.2858399	17.9409506
C	35.4036827	-4.6889806	0.61738469	-11.034357	8.41287971
CAT	10.0006154	9.30766809	9.92089095	8.03703771	11.2451069

CRM	10.0485906	10.8363509	8.00253161	-16.371046	48.3916149
CSCO	7.42166932	-1.4778082	37.8529936	-12.814152	3.2971023
CVX	11.0743322	-6.8102465	30.0199379	27.2589356	-12.203726
DIS	21.5955862	7.98819977	-9.2737801	-19.200922	-0.3695768
DOW	0.0547696	1.93646856	3.77418689	-4.2228401	6.76698079
GS	25.1712383	5.14918619	29.9733156	-4.9344526	6.97805876
HD	24.2828208	8.73821608	47.0456082	-11.741817	5.59428674
HON	31.0235102	7.84900383	-0.2399185	2.17637072	-4.0659503
IBM	16.1776254	-0.6116918	11.4085386	6.11110705	16.4313454
INTC	16.400627	-4.4557052	3.03768961	-19.700844	35.7198084
JNJ	13.2721827	5.43929177	12.5379883	4.09912797	-12.452303
JPM	37.446978	-1.7144033	20.6004417	-7.3841172	18.9402234
KO	16.8707651	0.92903461	11.8951208	7.33408611	-10.611021
MCD	12.2359392	4.29298387	28.8473469	-0.7276113	11.1086764
MMM	-3.987242	1.00723092	4.16420214	-17.870596	-4.6034771
MRK	16.9839904	-3.7643519	1.31064415	37.8539555	-3.1663317
MSFT	43.4325013	15.111926	39.5754938	-13.09519	32.4821803
NKE	26.981161	15.3849393	10.6076948	-11.318366	-6.1690898
PG	35.3930457	6.67107363	22.6206742	-4.6310539	-5.888251
TRV	13.9037223	1.61596449	10.2088616	14.4631431	-0.7076998
UNH	10.9630554	6.8203578	37.5064363	3.55848636	-2.9586416
V	35.4986422	6.17185233	-0.2109088	-2.4453984	21.0124148
VZ	11.4861602	-0.32978	-8.3802438	-14.884179	-1.418363
WMT	30.2297551	11.4671743	1.86999057	-1.1138189	7.78804097
Total	20.0746386	4.96513446	13.1536784	-2.7026581	8.90763885

However, the results shown in the above table are very different from those from Stata. We reviewed the Python code again and found that the standard deviation of the excess returns was incorrect because it was not annualized like the excess returns. Thus, we corrected the Python code as follows:

```

import pandas as pd

# Load the Dow 30 daily returns data
dow_data_path = 'path/to/Dow 30 Daily Returns.csv'
dow_data = pd.read_csv(dow_data_path)

# Load the Fama-French 3 factors data (including the risk-free rate)
ff_data_path = 'path/to/FamaFrench_3Factors.csv'
ff_data = pd.read_csv(ff_data_path)

# Convert date columns to datetime format
dow_data['date'] = pd.to_datetime(dow_data['date'])
ff_data['date'] = pd.to_datetime(ff_data['Date'], format='%Y%m%d')

# Merge the datasets on the date column
merged_data = pd.merge(dow_data, ff_data, on='date', how='inner')

# Calculate the excess return (raw return - risk free rate)
merged_data['excess_ret'] = merged_data['RET'] - merged_data['RF']

# Extract the year from the date for annual calculations
merged_data['year'] = merged_data['date'].dt.year

# Function to annualize returns
def annualize_return(daily_returns):
    return (1 + daily_returns).prod() ** (252 / len(daily_returns)) - 1

# Compute the annualized excess return for each stock each year
annual_excess_return = merged_data.groupby(['TICKER',
'year'])['excess_ret'].apply(annualize_return).reset_index()
annual_excess_return.columns = ['TICKER', 'year', 'annual_excess_return']

# Compute the standard deviation of excess returns for each stock each year
std_excess_return = merged_data.groupby(['TICKER', 'year'])['excess_ret'].std().reset_index()
std_excess_return['excess_ret'] = std_excess_return['excess_ret'] * (252 ** 0.5)
std_excess_return.columns = ['TICKER', 'year', 'std_excess_return']

# Merge the annualized return and standard deviation data
annual_stats = pd.merge(annual_excess_return, std_excess_return, on=['TICKER', 'year'])

# Calculate the Sharpe Ratio (annualized excess return / standard deviation of excess return)
annual_stats['sharpe_ratio'] = annual_stats['annual_excess_return'] /
annual_stats['std_excess_return']

# Calculate the average Sharpe Ratio for all stocks each year
average_sharpe_ratio = annual_stats.groupby('year')['sharpe_ratio'].mean().reset_index()
average_sharpe_ratio.columns = ['year', 'average_sharpe_ratio']

# Merge the average Sharpe Ratio with the annual stats
annual_stats = pd.merge(annual_stats, average_sharpe_ratio, on='year', how='left')

# Display the results
pd.set_option('display.max_columns', None)
print(annual_stats.to_string())
print(average_sharpe_ratio.to_string())

```

We obtained the following results by running the corrected code in Python:

TICKER	2019	2020	2021	2022	2023
AAPL	3.251206	1.739760	1.380334	-0.771323	2.090745
AMGN	1.191135	-0.062056	0.043803	0.865322	0.397591
AMZN	0.891517	1.951833	0.098600	-1.007690	2.216823
AXP	1.741743	-0.023946	1.318358	-0.276775	0.878734
BA	0.039458	-0.389415	-0.165680	-0.143994	1.130174
C	2.230222	-0.295378	0.038892	-0.695099	0.529962
CAT	0.629980	0.586328	0.624957	0.506286	0.708375
CRM	0.633002	0.682626	0.504112	-1.031279	3.048385
CSCO	0.467521	-0.093093	2.384514	-0.807216	0.207698
CVX	0.697617	-0.429005	1.891078	1.717152	-0.768762
DIS	1.360394	0.503209	-0.584193	-1.209544	-0.023281
DOW	0.003450	0.121986	0.237751	-0.266014	0.426280
GS	1.585639	0.324368	1.888141	-0.310841	0.439576
HD	1.529674	0.550456	2.963595	-0.739665	0.352407
HON	1.954297	0.494441	-0.015113	0.137098	-0.256131
IBM	1.019095	-0.038533	0.718670	0.384964	1.035077
INTC	1.033142	-0.280683	0.191356	-1.241037	2.250136
JNJ	0.836069	0.342643	0.789819	0.258221	-0.784421
JPM	2.358938	-0.107997	1.297706	-0.465156	1.193122
KO	1.062758	0.058524	0.749322	0.462004	-0.668431
MCD	0.770792	0.270433	1.817212	-0.045835	0.699781
MMM	-0.251173	0.063450	0.262320	-1.125742	-0.289992
MRK	1.069891	-0.237132	0.082563	2.384575	-0.199460
MSFT	2.735990	0.951962	2.493022	-0.824919	2.046185
NKE	1.699653	0.969160	0.668222	-0.712990	-0.388616
PG	2.229552	0.420238	1.424969	-0.291729	-0.370925
TRV	0.875852	0.101796	0.643098	0.911092	-0.044581
UNH	0.690608	0.429642	2.362683	0.224164	-0.186377
V	2.236204	0.388790	-0.013286	-0.154046	1.323658
VZ	0.723560	-0.020774	-0.527906	-0.937615	-0.089348
WMT	1.904296	0.722364	0.117798	-0.070164	0.490600
Total	1.264583	0.312774	0.828604	-0.170252	0.561129

Thus, the results from Python are very similar to those in Panel B generated using Stata. The small difference is due to the method of annualization. The code in Stata uses the actual number of trading days to compute the annual excess return and its annual standard deviation, for example, 252 days in 2019 and 253 days in 2020. However, the code suggested by ChatGPT-4o uses 252 days to annualize excess return in all years, regardless of the actual trading days. If we also use the actual trading days to compute the annualized return and standard deviation in Python, we would obtain the exact same results in both Stata and Python.

Table 7: Market Beta Analysis

This table assesses the market (CAPM) beta for each stock each year. Panel A presents the results generated using Stata. Panel B shows the results generated using ChatGPT-4o. The "Prompt" refers to the request made by the authors, while the "Response" refers to the feedback provided by ChatGPT-4o. Panel C offers an "Assessment" of the quality and accuracy of the ChatGPT-4o response provided by the authors.

Panel A: Results from Stata

TICKER	2019	2020	2021	2022	2023
AAPL	1.4751123	1.1174583	1.1956272	1.2385795	1.0368298
AMGN	0.68597156	0.78617704	0.5071367	0.31121698	0.47645012
AMZN	1.2759426	0.68658948	0.98217767	1.5921936	1.464787
AXP	1.008621	1.4660687	1.0471015	1.077038	1.1996084
BA	0.95004112	1.7258273	1.4960568	1.1925852	0.97405726
C	1.4439909	1.5658641	0.86605388	0.9147712	1.1263285
CAT	1.3869772	0.98232561	0.85957801	0.73104262	1.102708
CRM	1.2386572	1.0655986	1.1196494	1.4709156	1.1648153
CSCO	1.2299453	0.97472245	0.73084158	0.78048164	0.71736181
CVX	0.80189067	1.3591725	0.81278944	0.52070349	0.57260776
DIS	0.73806602	1.0486075	0.86295199	1.1206871	1.0726198
DOW	1.4848993	1.3272684	1.0509194	0.72340429	0.94973165
GS	1.2125224	1.2718841	0.91507107	0.91560948	1.0485996
HD	0.81915396	1.0915444	0.70227456	0.88494635	1.0125784
HON	1.0125303	1.0724968	0.78497875	0.73390293	0.8524366
IBM	1.0314893	0.9806186	0.4211342	0.48465607	0.47231638
INTC	1.2596713	1.1506979	1.262306	1.1338916	1.3475364
JNJ	0.49547896	0.65726101	0.2881622	0.26355907	0.29711422
JPM	1.020825	1.2737336	0.75167328	0.82942206	0.84921855
KO	0.38498557	0.78940171	0.40562904	0.43669266	0.34200588
MCD	0.36507782	0.93903095	0.5414601	0.44058502	0.45249423
MMM	1.1402873	0.80912733	0.36016268	0.63740867	1.1587117
MRK	0.59414691	0.68459654	0.19001053	0.24593455	0.21484913
MSFT	1.2183875	1.1175761	1.0304406	1.2136495	1.0744908
NKE	1.0697894	0.94029278	0.91890192	1.2785211	0.94393331
PG	0.46374574	0.68833619	0.23536041	0.41887146	0.33232403
TRV	0.61573184	1.0550519	0.72342438	0.39097619	0.48993209
UNH	0.69263011	1.1224691	0.50640661	0.51883662	0.23741463
V	1.000208	1.1191038	0.92548811	0.91953123	0.72169179
VZ	0.32166165	0.51305872	0.13739157	0.33252576	0.44305083
WMT	0.463933	0.502122	0.411849	0.402764	0.363042

Panel B: Results from ChatGPT-4o

Prompts:	
[upload from computer the "Dow 30 Daily Returns" file & the "FamaFrench 3Factors" file]	
Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS, and the daily risk free rate, pulled from Kenneth R. French - Data Library. Compute the CAPM's Beta for each stock each year.	
Response:	
It seems there's an issue with inspecting the data columns. I will provide you with a detailed guide on how to analyze this data to compute the CAPM's Beta for each stock each year on your local machine.	
Steps to Compute CAPM's Beta	
Load and Merge Data: Load the Dow 30 returns data and the Fama-French 3 factors data, then merge them. Calculate Excess Returns: Calculate the excess returns for each stock and the market. Compute Beta: Compute the CAPM's Beta for each stock for each year using linear regression.	
Detailed Guide	
1. Load and Merge Data	
Python	Copy code
<pre>import pandas as pd # Load the Dow 30 daily returns data dow_data_path = 'path/to/Dow 30 Daily Returns.csv' dow_data = pd.read_csv(dow_data_path) # Load the Fama-French 3 factors data ff_data_path = 'path/to/FamaFrench_3Factors.csv' ff_data = pd.read_csv(ff_data_path) # Convert date columns to datetime format dow_data['date'] = pd.to_datetime(dow_data['date']) ff_data['Date'] = pd.to_datetime(ff_data['Date'], format='%Y%m%d') # Merge the datasets on the date column merged_data = pd.merge(dow_data, ff_data, left_on='date', right_on='Date', how='inner')</pre>	
2. Compute Excess Returns	
Python	Copy code
<pre># Calculate the excess return for each stock and the market merged_data['excess_stock_ret'] = merged_data['RET'] - merged_data['RF'] merged_data['excess_market_ret'] = merged_data['Mkt-RF']</pre>	
3. Compute Beta	
Python	Copy code
<pre># Function to compute Beta using linear regression def compute_beta(group): slope, intercept, r_value, p_value, std_err = stats.linregress(group['excess_market_ret'], group['excess_stock_ret']) return slope # Extract the year from the date for annual calculations merged_data['year'] = merged_data['date'].dt.year # Group data by TICKER and year to compute Beta for each stock each year</pre>	

	<pre> betas = merged_data.groupby(['TICKER', 'year']).apply(compute_beta).reset_index() betas.columns = ['TICKER', 'year', 'beta'] # Display the results print(betas.head()) </pre>	
Complete Script		
	Python	Copy code
<pre> import pandas as pd from scipy import stats # Load the Dow 30 daily returns data dow_data_path = 'path/to/Dow 30 Daily Returns.csv' dow_data = pd.read_csv(dow_data_path) # Load the Fama-French 3 factors data ff_data_path = 'path/to/FamaFrench_3Factors.csv' ff_data = pd.read_csv(ff_data_path) # Convert date columns to datetime format dow_data['date'] = pd.to_datetime(dow_data['date']) ff_data['Date'] = pd.to_datetime(ff_data['Date'], format='%Y%m%d') # Merge the datasets on the date column merged_data = pd.merge(dow_data, ff_data, left_on='date', right_on='Date', how='inner') # Calculate the excess return for each stock and the market merged_data['excess_stock_ret'] = merged_data['RET'] - merged_data['RF'] merged_data['excess_market_ret'] = merged_data['Mkt-RF'] # Function to compute Beta using linear regression def compute_beta(group): slope, intercept, r_value, p_value, std_err = stats.linregress(group['excess_market_ret'], group['excess_stock_ret']) return slope # Extract the year from the date for annual calculations merged_data['year'] = merged_data['date'].dt.year # Group data by TICKER and year to compute Beta for each stock each year betas = merged_data.groupby(['TICKER', 'year']).apply(compute_beta).reset_index() betas.columns = ['TICKER', 'year', 'beta'] # Display the results print(betas.head()) </pre>		
By following these steps on your local machine, you should be able to compute the CAPM's Beta for each stock each year. If you encounter any specific issues or errors, please provide the details so I can assist you further. [View Analysis]		

Panel C: Assessment of the Results from ChatGPT-4o

We could not obtain results using the initial Python code provided by ChatGPT-4o. There are a few errors. First, the variable name for "excess market return" in the merged_data dataset should be 'MktRF', not 'Mkt-RF'. Second, the beta value for DOW in 2019 was missing because the value of the 'ret' variable was null on April 2, 2019. We corrected these issues by renaming the variable and adding a line of code to delete observations with null values in either the 'excess_stock_ret' or 'excess_market_ret' columns.

```

import pandas as pd
from scipy import stats

# Load the Dow 30 daily returns data
dow_data_path = 'path/to/Dow 30 Daily Returns.csv'
dow_data = pd.read_csv(dow_data_path)

# Load the Fama-French 3 factors data
ff_data_path = 'path/to/FamaFrench_3Factors.csv'
ff_data = pd.read_csv(ff_data_path)

# Convert date columns to datetime format
dow_data['date'] = pd.to_datetime(dow_data['date'])
ff_data['Date'] = pd.to_datetime(ff_data['Date'], format='%Y%m%d')

# Merge the datasets on the date column
merged_data = pd.merge(dow_data, ff_data, left_on='date', right_on='Date', how='inner')

# Calculate the excess return for each stock and the market
merged_data['excess_stock_ret'] = merged_data['RET'] - merged_data['RF']
merged_data['excess_market_ret'] = merged_data['MktRF']
merged_data.dropna(subset=['excess_stock_ret', 'excess_market_ret'], inplace=True)

# Function to compute Beta using linear regression
def compute_beta(group):
    slope, intercept, r_value, p_value, std_err = stats.linregress(group['excess_market_ret'],
group['excess_stock_ret'])
    return slope

# Extract the year from the date for annual calculations
merged_data['year'] = merged_data['date'].dt.year

# Group data by TICKER and year to compute beta for each stock each year
betas = merged_data.groupby(['TICKER', 'year']).apply(compute_beta).reset_index()
betas.columns = ['TICKER', 'year', 'beta']

# Display the results
print(betas.head())
print(betas.to_string())

```

After fixing these issues, the results obtained were consistent with those from Stata.

TICKER	2019	2020	2021	2022	2023
AAPL	1.475112	1.117458	1.195627	1.238580	1.036830
AMGN	0.685972	0.786177	0.507137	0.311217	0.476450
AMZN	1.275943	0.686589	0.982178	1.592194	1.464787
AXP	1.008621	1.466069	1.047102	1.077038	1.199609
BA	0.950041	1.725827	1.496057	1.192585	0.974057
C	1.443991	1.565864	0.866054	0.914771	1.126328
CAT	1.386977	0.982326	0.859578	0.731043	1.102708
CRM	1.238657	1.065599	1.119649	1.470916	1.164815

CSCO	1.229945	0.974722	0.730842	0.780482	0.717362
CVX	0.801891	1.359172	0.812789	0.520704	0.572608
DIS	0.738066	1.048607	0.862952	1.120687	1.072620
DOW	1.484899	1.327268	1.050919	0.723404	0.949732
GS	1.212522	1.271884	0.915071	0.915609	1.048600
HD	0.819154	1.091544	0.702275	0.884946	1.012578
HON	1.012530	1.072497	0.784979	0.733903	0.852437
IBM	1.031489	0.980619	0.421134	0.484656	0.472316
INTC	1.259671	1.150698	1.262306	1.133892	1.347536
JNJ	0.495479	0.657261	0.288162	0.263559	0.297114
JPM	1.020825	1.273734	0.751673	0.829422	0.849219
KO	0.384986	0.789402	0.405629	0.436693	0.342006
MCD	0.365078	0.939031	0.541460	0.440585	0.452494
MMM	1.140287	0.809127	0.360163	0.637409	1.158712
MRK	0.594147	0.684597	0.190011	0.245935	0.214849
MSFT	1.218387	1.117576	1.030441	1.213650	1.074491
NKE	1.069789	0.940293	0.918902	1.278521	0.943933
PG	0.463746	0.688336	0.235360	0.418871	0.332324
TRV	0.615732	1.055052	0.723424	0.390976	0.489932
UNH	0.692630	1.122469	0.506407	0.518837	0.237415
V	1.000208	1.119104	0.925488	0.919531	0.721692
VZ	0.321662	0.513059	0.137392	0.332526	0.443051
WMT	0.463933	0.502122	0.411849	0.402764	0.363042

Table 8: ARMA-GARCH Estimation

This table assesses the ARMA-GARCH estimation for the market return (vwretd). Panel A presents the results generated using Stata. Panel B shows the results generated using ChatGPT-4o. The "Prompt" refers to the request made by the authors, while the "Response" refers to the feedback provided by ChatGPT-4o. Panel C offers an "Assessment" of the quality and accuracy of the ChatGPT-4o response provided by the authors.

Panel A: Results from Stata

	vwretd	
vwretd		
Intercept	0.001 ***	
	(0.000)	
ARMA		
L.ar	-0.581	
	(0.496)	
L.ma	0.525	
	(0.518)	
ARCH		
L.arch	0.424 ***	
	(0.036)	
L.garch	0.637 ***	
	(0.060)	
Intercept	-0.000	
	(0.000)	
Number of observations	1258	
Log likelihood	3829.98	
*** p<.01, ** p<.05, * p<.1		

Panel B: Results from ChatGPT-4o

Prompts:	
[upload from computer the "Dow 30 Daily Returns" file]	
Act as a finance professor and statistician. Analyze this data for the daily returns data of Dow 30 components from 2019 to 2023, pulled from WRDS. Estimate an ARMA-GARCH model on the value-weighted return (vwretd).	
Response:	
It seems there's an issue with executing the model fitting directly. I'll provide a detailed guide on how to estimate an ARMA-GARCH model on the value-weighted return (vwretd) using Python on your local machine.	
Steps to Estimate ARMA-GARCH Model	
1.	Load the Data: Load the Dow 30 returns data and set up the time series.
2.	Estimate ARMA Model: Fit an ARMA model to the value-weighted return.
3.	Estimate GARCH Model: Fit a GARCH model on the residuals of the ARMA model.
Detailed Guide	
1. Load and Merge Data	
Python	Copy code
import pandas as pd	
# Load the Dow 30 daily returns data	
dow_data_path = 'path/to/Dow 30 Daily Returns.csv'	
dow_data = pd.read_csv(dow_data_path)	

<pre> # Convert date column to datetime format dow_data['date'] = pd.to_datetime(dow_data['date']) # Set date as index for time series analysis dow_data.set_index('date', inplace=True) # Extract the value-weighted return (vwretd) vwretd = dow_data['vwretd'].dropna() </pre>	
2. Estimate ARMA Model	
Python	Copy code
<pre> import statsmodels.api as sm # Estimate ARMA model (start with ARMA(1,1) as an example) arma_model = sm.tsa.ARMA(vwretd, order=(1, 1)).fit() # Summarize the ARMA model results arma_summary = arma_model.summary() print(arma_summary) </pre>	
3. Estimate GARCH Model	
Python	Copy code
<pre> from arch import arch_model # Estimate GARCH model on the residuals of the ARMA model garch_model = arch_model(arma_model.resid, vol='Garch', p=1, q=1) garch_fit = garch_model.fit(dispatch='off') # Summarize the GARCH model results garch_summary = garch_fit.summary() print(garch_summary) </pre>	
Complete Script	
Python	Copy code
<pre> import pandas as pd import statsmodels.api as sm from arch import arch_model # Load the Dow 30 daily returns data dow_data_path = 'path/to/Dow 30 Daily Returns.csv' dow_data = pd.read_csv(dow_data_path) # Convert date column to datetime format dow_data['date'] = pd.to_datetime(dow_data['date']) # Set date as index for time series analysis dow_data.set_index('date', inplace=True) # Extract the value-weighted return (vwretd) vwretd = dow_data['vwretd'].dropna() # Estimate ARMA model (start with ARMA(1,1) as an example) arma_model = sm.tsa.ARMA(vwretd, order=(1, 1)).fit() </pre>	

<pre># Summarize the ARMA model results arma_summary = arma_model.summary() print(arma_summary) # Estimate GARCH model on the residuals of the ARMA model garch_model = arch_model(arma_model.resid, vol='Garch', p=1, q=1) garch_fit = garch_model.fit(dispatch='off') # Summarize the GARCH model results garch_summary = garch_fit.summary() print(garch_summary)</pre>	
By following these steps on your local machine, you should be able to estimate the ARMA-GARCH model on the value-weighted return (vwret). If you encounter any specific issues or errors, please provide the details so I can assist you further. [View Analysis]	

Prompts:	
Python code has the following errors: Traceback (most recent call last): File "/Users/fliu/PycharmProjects/helloworld/chatGPT531/Table 6 test.py", line 21, in <module> arma_model = sm.tsa.ARMA(vwret, order=(1, 1)).fit() ^^^^^^^^^^ AttributeError: module 'statsmodels.tsa.api' has no attribute 'ARMA'. Did you mean: 'ARIMA'?	
Response:	
The error you encountered is because the ARMA class has been deprecated and is replaced by the ARIMA class in the statsmodels library. You should use the ARIMA class with the appropriate order for ARMA models. Here is the corrected code using the ARIMA class for ARMA model estimation: Corrected Script	
Python <pre>import pandas as pd import statsmodels.api as sm from arch import arch_model # Load the Dow 30 daily returns data dow_data_path = 'path/to/Dow 30 Daily Returns.csv' dow_data = pd.read_csv(dow_data_path) # Convert date column to datetime format dow_data['date'] = pd.to_datetime(dow_data['date']) # Set date as index for time series analysis dow_data.set_index('date', inplace=True) # Extract the value-weighted return (vwret) vwret = dow_data['vwret'].dropna() # Estimate ARMA model (start with ARIMA(1,0,1) as an example for ARMA(1,1)) arma_model = sm.tsa.ARIMA(vwret, order=(1, 0, 1)).fit()</pre>	Copy code

<pre># Summarize the ARMA model results arma_summary = arma_model.summary() print(arma_summary) # Estimate GARCH model on the residuals of the ARMA model garch_model = arch_model(arma_model.resid, vol='Garch', p=1, q=1) garch_fit = garch_model.fit(dispatch='off') # Summarize the GARCH model results garch_summary = garch_fit.summary() print(garch_summary)</pre>	Explanation 1. Loading the Data: Load the data and preprocess it by setting the date as the index. 2. Estimating the ARMA Model: Use ARIMA with the order (1, 0, 1) to specify an ARMA(1,1) model. The ARIMA function allows specifying the AR and MA components directly. 3. Estimating the GARCH Model: Fit a GARCH(1,1) model on the residuals of the ARMA model using the arch package. By following this corrected script, you should be able to estimate the ARMA-GARCH model on the value-weighted return (vwret) without encountering the AttributeError. If you encounter any further issues or need additional assistance, please let me know!
--	---

Panel C: Assessment of the Results from ChatGPT-4o

We didn't obtain any results using the initial Python code provided by ChatGPT-4o due to a few errors. First, the 'ARMA' function in the 'statsmodels.tsa.api' module has been replaced by the 'ARIMA' function. Thus, we should use ARIMA(1,0,1) instead of ARMA(1,1). Second, our data is a panel data of 30 firms over 5 years, so the 'vwret' values must be repeated 30 times in the data. Thus, we can simply keep observations for just one stock and delete duplicate data. To address this, we added a line of code: "dow_data = dow_data[dow_data['TICKER'] == 'AAPL']." Thus, the corrected code is as follows.

Python	Copy code
<pre>import pandas as pd import statsmodels.api as sm from arch import arch_model # Load the Dow 30 daily returns data dow_data_path = 'path/to/Dow 30 Daily Returns.csv' dow_data = pd.read_csv(dow_data_path) # Convert date column to datetime format dow_data['date'] = pd.to_datetime(dow_data['date']) dow_data = dow_data[dow_data['TICKER'] == 'AAPL'] # Set date as index for time series analysis dow_data.set_index('date', inplace=True) # Extract the value-weighted return (vwret) vwret = dow_data['vwret'].dropna()</pre>	

```

# Estimate ARMA model (start with ARIMA(1,0,1) as an example for ARMA(1,1))
arma_model = sm.tsa.ARIMA(vwretd, order=(1, 0, 1)).fit()

# Summarize the ARMA model results
arma_summary = arma_model.summary()
print(arma_summary)

# Estimate GARCH model on the residuals of the ARMA model
garch_model = arch_model(arma_model.resid, vol='Garch', p=1, q=1)
garch_fit = garch_model.fit(dispatch='off')

# Summarize the GARCH model results
garch_summary = garch_fit.summary()
print(garch_summary)

```

We obtained the following results using the corrected Python code. The first table below shows the estimated results for the ARIMA (1,0,1) model, and the second table shows the results for the GARCH(1,1) model. We can clearly see that these results differ from those given by Stata. The reason is that Stata estimates ARIMA and GARCH models simultaneously, while Python estimates these two models sequentially. Unfortunately, there is currently no package in Python that can estimate these two models jointly.

ARIMA (1,0,1)		
vwretd		
Intercept	0.0006	*
	(0.000)	
L.ar	-0.5738	***
	(0.051)	
L.ma	0.4299	***
	(0.055)	
Number of observations	1258	
Log likelihood	3671.892	
*** p<.01, ** p<.05, * p<.1		

GARCH		
sigma^2		
Omega	3.4142e-06	***
	(2.422e-11)	
alpha(1)	0.2000	***
	(0.03641)	
beta(1)	0.7800	***
	(0.02977)	
Number of observations	1258	
Log likelihood	3979.10	
*** p<.01, ** p<.05, * p<.1		